



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

Факултет Приложна математика и информатика



ДИПЛОМНА РАБОТА

Тема:

**Методи за детекция на финансови измами чрез
машинно обучение при небалансирани данни**

Дипломант: Тодор Георгиев Стаматов

Фак. №: 961324007

Специалност: Анализ на големи масиви и потоци данни

Образователно- квалификационна степен: Магистър.

Дипломен ръководител: Проф. дн. Пламенка Боровска
София 2026



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

Факултет Приложна математика и информатика



Утвърждавам:

Декан на ФПМИ:

(Доц. д-р инж. Д. Иванова)

Дата на задаване: 15.01.2025

ЗАДАНИЕ

за разработка на дипломна работа

Дипломант: Тодор Георгиев Стаматов

Фак. №: 961324007

Специалност: Анализ на големи масиви и потоци данни

Образователно-квалификационна степен: Магистър

1. *Тема на дипломната работа:*

Методи за детекция на финансови измами чрез машинно обучение при небалансирани данни

2. *Заявител на темата:* Катедра Информатика - ФПМИ

3. *Обяснителна записка :*

Дипломната работа разглежда приложението на алгоритми за машинно обучение при детекция на финансови измами в транзакционни данни. Основен акцент е поставен върху анализа на небалансирани набори от данни, характерни за задачите по откриване на измами, при които измамните транзакции представляват малък процент от всички наблюдения. В рамките на изследването ще бъдат разгледани и сравнени различни алгоритми за класификация, като Logistic Regression, Random Forest, Support Vector Machines и градиентно буустване. Ще бъдат приложени и методи за справяне с небалансирани данни, включително техники за преизвадка и оценка чрез подходящи метрики като precision, recall, F1-score и ROC-AUC. Практическата част включва обработка и анализ на публично достъпни транзакционни данни, обучение на модели и сравнителна оценка на тяхната ефективност. Целта на работата е да се идентифицират подходящи подходи за надеждна и ефективна автоматична детекция на финансови измами.

Дипломен ръководител:

Ръководител катедра:

СЪДЪРЖАНИЕ

АБСТРАКТ

ИЗЛОЖЕНИЕ

Въведение

Преглед на литературата

Клинични методи за диагностика

Съществуващи изследвания с помощта на машинни алгоритми за редки
чернодробни болести

Изследване на структурата на данните

Предварителна обработка на данните

Базови модели и първични резултати

Random Forest

Логистична регресия (Logistic Regression)

Support Vector Machine (SVM)

Анализирани метрики

Резултати от Random Forest

Резултати от Логистична регресия

Резултати от Support Vector Machine

Обобщение на резултатите

Тест за устойчивост - добавяне на шум

Резултати от Random Forest

Резултати от Логистична регресия

Резултати от Support Vector Machine

Обобщение на резултатите

Модели, работещи с непълни данни

LightGBM

XGBoost

Резултати от LightGBM

Резултати от XGBoost

Обобщение на резултатите

ЗАКЛЮЧЕНИЕ

ИЗПОЛЗВАНИ ЛИТЕРАТУРНИ ИЗТОЧНИЦИ

ИЗХОДЕН КОД НА ИЗСЛЕДВАНЕТО

АБСТРАКТ

Настоящата дипломна работа има за цел да изследва приложението на съвременни алгоритми за машинно обучение при автоматизирана детекция на финансови измами в контекста на силно небалансирани транзакционни данни — типичен сценарий в банковия и платежния сектор, където измамните операции са рядко срещани, но с висок икономически и социален ефект.

Работата включва събиране и първичен анализ на набор от данни, предварителна обработка на транзакционната информация, избор и прилагане на подходящи модели на машинно обучение, както и анализ на точността и ефективността на всеки модел при оценка, адекватна за рядък положителен клас. Наборът от данни е PaySim — синтетичен, т.е. изкуствено генериран симулатор на мобилни парични преводи, стремящ се максимално близко да отразява поведението на реални финансови транзакции. Съдържа атрибути като времеви стъпка, тип операция, сума, баланси на изпращача и получателя, както и етикети за измама и маркиране от системата.

Основните задачи включват:

- изследване на структурата на данните и разпределението на класовете (EDA);
- предварителна обработка и кодиране на категориални признаци;
- анализ на корелациите и прилагане на техники за намаляване на размерността (PCA);
- прилагане на стратегии за работа с небалансирани данни (тегла на класовете, SMOTE, undersampling) с внимание към избягване на изтичане на информация между train и validation множествата;
- обучение на различни модели (като Logistic Regression, Random Forest, Support Vector Machine, LightGBM, XGBoost и др.);
- оценка на производителността чрез крос-валидация с метрики ROC AUC, PR AUC и F1.

Целта е да се идентифицират най-подходящите подходи за автоматизирано разпознаване на измамни транзакции въз основа на поведенчески и финансови показатели. Това има потенциал за по-ранно и по-точно откриване на измамна дейност, което е критично за намаляване на загубите и защита на клиентите и институциите. Разработката доказва приложимостта на алгоритмите за машинно обучение при

детекция на финансови измами при екстремна класова небалансираност и предоставя основа за бъдещи практически и научни приложения.

Работата е реализирана в програмна среда Python с използване на библиотеки като Pandas, Scikit-learn, XGBoost, LightGBM, imbalanced-learn, Matplotlib, Seaborn и др. Анализите и визуализациите са проведени в Jupyter Notebook (scams-detection.ipynb). Допълнително е изградено React уеб приложение (dashboard) за интерактивна визуализация на агрегирани данни от PaySim и на резултатите от обучението на модели, което улеснява представянето и сравнението на експериментите.

ИЗЛОЖЕНИЕ

Въведение

Финансовите измами представляват сериозен и нарастващ риск за банки, платежни системи и доставчици на цифрови финансови услуги. Според отчети на индустрията и академичната литература загубите от измамни транзакции достигат милиарди долари годишно, а обемът на електронните плащания непрекъснато нараства. Типичният проблем при автоматизирана детекция е силната класова небалансираност: легитимните операции са многократно повече от измамните, което прави метрики като общата точност (ассигасу) подвеждащи и изисква специализирани подходи при обучение и оценка на моделите.

В настоящата работа се използва наборът от данни PaySim — синтетичен симулатор на мобилни парични преводи, базиран на реално поведение на транзакции и публикуван за изследвания в областта на fraud detection. Данните съдържат милиони записи с атрибути като времеви стъпка (step), тип операция (PAYMENT, TRANSFER, CASH_IN, CASH_OUT, DEBIT), сума, баланси на изпращача и получателя, както и етикети isFraud и isFlaggedFraud. Дялът на измамните транзакции е изключително нисък (порядък 0,1–0,2%), което реалистично отразява условията в системите за мониторинг.

Класическите правила и филтри често не улавят сложни или нови схеми на измама. Моделите за машинно обучение могат да откриват аномалии във финансовите потоци — например несъответствия в балансите, необичайни суми или комбинации от тип транзакция и поведение на сметките — но изискват внимателна предварителна обработка, коректна валидация без изтичане на информация между train и test множествата и метрики, подходящи за рядък положителен клас (ROC AUC, PR AUC, F1, recall).

Ранното и надеждно откриване на измамни транзакции е от изключителна важност за намаляване на финансовите загуби, защита на клиентите и поддържане на доверието в платежните услуги. Пропускът на реална измама (нисък recall) обикновено е по-скъп от фалшива тревога; затова в работата се анализират не само точността на класификацията, но и компромисът между precision и recall при различни стратегии за работа с небалансирани данни.

Задачите за изследването включват:

1. Подробен анализ и преглед на структурата на данните. Визуализиране на разпределението на различните характеристики и техните взаимоотношения. Изследване на формата на данните — кои характеристики са числови и кои категорийни. Анализ на дела измами спрямо легитимни транзакции и на разпределението по тип операция. Проверка за липсващи стойности и основни статистики.
2. Предварителна обработка на набора от данни. Преобразуване на категорийните характеристики в числови (one-hot encoding). Калкулиране на корелационна матрица между характеристиките. Намалване на размерността и визуализация на взаимоотношението между измамни и легитимни транзакции чрез Principal Component Analysis (PCA).
3. Прилагане на стратегии за небалансирани класове (baseline — без корекция). Обучение на модели за машинно обучение върху данните без изкуствена корекция на разпределението на класовете. Крос-валидация със StratifiedKFold. Анализ на резултатите чрез ROC AUC, PR AUC, F1, precision и recall. Сравнение на алгоритми като Logistic Regression, Random Forest и Support Vector Machine.
4. Прилагане на тегла на класовете (class_weight). Обучение на същите модели с корекция на загубата/тежестите в полза на малцинствения клас. Калкулиране на корелационната карта и PCA върху обработените признаци. Анализ на производителността и на най-значимите характеристики за всеки модел (където алгоритъмът го позволява).
5. Синтетично овърсемплиране с SMOTE. Прилагане на SMOTE само върху train fold след разделяне, за да се избегне изтичане на данни. Трениране и оценка на моделите при увеличен брой синтетични примери от класа „измама“. Анализ на компромиса precision/recall спрямо baseline подходите.
6. Undersampling на мнозинствения клас. Прилагане на RandomUnderSampler върху обучителното множество в рамките на крос-валидацията. Трениране на моделите и сравнителен анализ на резултатите с останалите стратегии.
7. Обучение на модели с gradient boosting. Трениране и оценка на LightGBM и XGBoost при различните стратегии за небалансираност. Сравнителен анализ на най-добрите комбинации „модел + стратегия“ и формулиране на заключения за приложимостта им при детекция на финансови измами.

8. Визуализация и представяне на резултатите. Експорт на обобщени ML резултати за React dashboard, за интерактивно представяне на агрегирани PaySim статистики и сравнение на експериментите.

Преглед на литературата

Класически методи за детекция на финансови измами

Финансовите измами в цифровите платежни системи често не се проявяват като единични „явни“ събития, а като серия от на пръв поглед нормални транзакции — преводи, тегления и плащания, които в комбинация разкриват злоупотреба. Заради огромния обем операции и крайно ниския дял на реалните измами (често под 1%) първоначално е вероятно подозрение да падне върху технически грешки, лимити или типично клиентско поведение, което забавя идентификацията. Понеже пропуснатата измама носи директни финансови загуби и репутационен риск, ранното откриване е от съществено значение.

В системите от реални свят традиционно се използват правила и прагови филтри (rule-based monitoring): например блокиране при необичайно голяма сума, многократни преводи за кратък период, несъответствие между баланс преди/след транзакцията или операции към „рискови“ получатели. В наборът PaySim този подход е частично моделиран чрез полето isFlaggedFraud — вградена евристика на симулатора, която маркира малка част от измамните събития, но не покрива по-голямата част от реалните измами (isFraud), което илюстрира ограниченията на статичните правила.

Статистически и поведенчески профили допълват правилата: анализ на честота, средни суми, отклонения от индивидуална „норма“ на клиента, геолокация, устройство, време на деня. При мобилни пари и FinTech средата тези сигнали се комбинират в скорингови карти и нива на риск (low / medium / high), след което случаите се насочват към ръчен преглед от анализатори по сигурност (fraud analysts).

Ръчна експертиза и разследване остават задължителни при високорискови аларми: проверка на история на сметката, KYC/AML данни, връзки между сметки и обратни транзакции. Този процес е бавен и скъп при милиони записи дневно, което мотивира автоматизацията.

От лабораторния (в смисъл на аналитичен) аспект се изследват: тип транзакция (PAYMENT, TRANSFER, CASH_IN, CASH_OUT, DEBIT), сума, баланси на изпращач и получател, времеви стъпки и мрежови връзки между сметки. Аномалии в запазването на баланса (несъответствие между стар баланс, сума и нов баланс) са чест сигнал за

симулирана или манипулирана операция — подход, използван и в настоящия проект чрез инженерство на признаци.

Машинно обучение в литературата се позиционира като допълнение или замяна на част от правилата: моделите учат нелинейни комбинации от признаци и могат да адаптират прагове, но изискват коректна работа с небалансирани класове и метрики извън суровата точност— тема, която заема централно място в настоящата работа.

Съществуващи изследвания с помощта на машинни алгоритми за детекция на финансови измами

По-долу е кратък преглед на представителни публикации, които използват машинно обучение за откриване на измами във финансови транзакции — включително набори с екстремна небалансираност, близки до PaySim.

#	Контекст / набор	ML метод(и)	Обем (n)	Основни признаци / фийчъри	Най-добра метрика (докладвана)	Източник
1	PaySim — симулатор на мобилни пари	Agent-Based Simulation (не класификатор)	~23–24 млн транзакции на run	тип, сума, баланси, агенти; валидация срещу реални логове (SSE)	Синтетичните данни са „толкова prudent колкото оригиналните“ за изследвания	López-Rojas, Elmir, Axelsson — PaySim: A financial mobile money simulator for fraud detection (EMSS, 2016)
2	PaySim — supervised fraud detection	LR, KNN, Decision Tree, Random Forest (+ class weights)	PaySim (страт. 80/20 split)	amount, balances, type, feature engineering	DT: F1 = 0,8018, Recall = 0,9696, ROC-AUC = 0,9845; RF: Recall = 0,9838, ROC-AUC =	Fraud Detection in Financial Transactions Using Machine Learning: Insights from

					0,9990 (нисък precision)	the PaySim Mobile Money Dataset (WJARR, 2025)
3	Кредитни карти (European Fraud)	RF, Logistic Regression, ANN; cost-sensitive / sampling	284 807 транзакции (492 измами)	PCA-трансформирани 28 признака (поверителност)	Подчертава PR криви и калибрация на праг; accuracy е подвеждаща при ~0,17% измами	Dal Pozzolo et al. — Calibrating Probability with Undersampling for Unbalanced Classification (IEEE ICDM / свързани работи)
4	Онлайн финансови транзакции (силен дисбаланс)	SMOTE + soft-voting ensemble (RF, XGBoost, LightGBM, AdaBoost)	European Credit Card Fraud	ANOVA F-test feature selection + SMOTE	Recall = 0,962, F1 = 0,864, AUC = 0,982, MCC = 0,868	Fraud Identification in Online Financial Transactions Combining SMOTE and Ensemble Classifiers (ETASR, 2025 — SMOVO)

Таб.1 Сравнение на научни изследвания за детекция на финансови измами при небалансирани данни

Систематични прегледи (напр. Ramírez-Alpizar et al., 2024 — Financial fraud detection through the application of machine learning techniques: a literature review, Nature Humanities and Social Sciences Communications) обобщават, че Random Forest, XGBoost и LightGBM са сред най-често използваните модели върху таблични транзакционни

данни; PaySim е един от най-цитираните публични заместители на реални банкови логове заради липсата на отворени производствени datasets.

Обобщение и мотивация за настоящата работа

След сравнителния преглед могат да се формулират следните изводи:

- Най-често използваните модели при транзакционни измами са дървовидни ансамбли и gradient boosting (Random Forest, XGBoost, LightGBM), приложени върху таблични признаци — сума, баланси, тип операция, време. Те постигат висок recall и ROC-AUC, но често с компромис по precision (много фалшиви аларми), което е критично за оперативна употреба.
- Небалансираността е централен проблем: в PaySim делът на измамите е около 0,1–0,2%, аналогично на кредитни карти и мобилни пари. Литературата препоръчва F1, precision–recall анализ, PR AUC, cost-sensitive обучение и внимателен resampling — не accuracy като единствена метрика.
- Ограничения в досегашните работи: много публикации сравняват една стратегия за баланс (напр. само class weights или само SMOTE), използват единичен train/test split вместо строга крос-валидация, или прилагат SMOTE преди разделяне на данните — риск от изтичане на данни. Често липсва систематично сравнение на none, class_weight, SMOTE и undersampling при еднакви модели и метрики. PaySim-симулаторът (ред 1 в таблицата) решава проблема с достъп до данни, но не предлага готов ML pipeline.
- С оглед на тези пропуски, настоящата дипломна работа има за цел да изследва методи за детекция на финансови измами при небалансирани данни върху PaySim: EDA → preprocessing → корелации → PCA → обучение на LR, RF, SVM, LightGBM и XGBoost при четири стратегии за дисбаланс, с StratifiedKFold, resampling само върху train fold и оценка чрез ROC AUC, PR AUC и F1. Работата не само сравнява алгоритми, а демонстрира как изборът на стратегия за небалансираност пряко променя компромиса precision/recall — извод с практическо значение за реални fraud detection системи. Резултатите се визуализират допълнително чрез React dashboard, което улеснява представянето и сравнението на експериментите.

Изследване на структурата на данните

Като за начало на изследването е добре да бъде разгледана структурата на набора от данни. За целта данните се зареждат в програмната среда и се прави анализ, за да бъдат добре разбрани.

Започва се с визуализация на първите 5 реда от данните - така се разбира какви колони (характеристики) и какви стойности са налични.

	step	type	amount	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	47	PAYMENT	5661.379883	101806.0	96144.617188	M1724774327	0.0	0.0	0	0
1	550	PAYMENT	9033.589844	0.0	0.000000	M1303408874	0.0	0.0	0	0
2	399	PAYMENT	40710.800781	43871.0	3160.199951	M1262896233	0.0	0.0	0	0
3	255	PAYMENT	9359.980469	0.0	0.000000	M2144867138	0.0	0.0	0	0
4	235	PAYMENT	3850.370117	0.0	0.000000	M9504230	0.0	0.0	0	0

Фиг. 1 Преглед на данните

Разглеждаме всяка характеристика от набора с данни:

Колона	Описание
step	Времени индекс на симулацията — една стъпка $\approx$ 1 час реално време (в оригиналния PaySim $\sim$ 744 стъпки = 1 месец).
type	Тип транзакция: PAYMENT, TRANSFER, CASH_IN, CASH_OUT или DEBIT.
amount	Сума на транзакцията (в валутата на симулатора).
oldbalanceOrg	Баланс на сметката на изпращача преди транзакцията.
newbalanceOrig	Баланс на сметката на изпращача след транзакцията.

Колона	Описание
nameDest	Идентификатор на получателя (в PaySim сметки на търговци започват с M). В notebook-а се ползва за признак „търговец“, не като директен ML признак.
oldbalanceDest	Баланс на сметката на получателя преди транзакцията.
newbalanceDest	Баланс на сметката на получателя след транзакцията.
isFraud	Целева променлива: 1 = измамна транзакция (симулирана от „злонамерен“ агент), 0 = легитимна.
isFlaggedFraud	Дали вградената rule-based система на PaySim е маркирала транзакцията; улавя само малка част от реалните измами.

Таб. 2 Описание на характеристиките

Следва преглед на типовете на характеристиките и броя на липсващи (null) стойности за всяка характеристика.

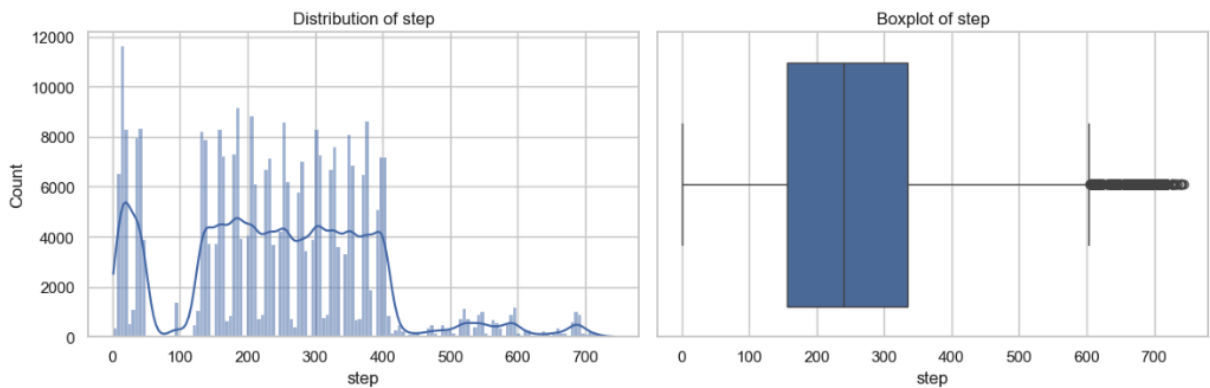
Колона	Тип в pandas	Категория	Брой null стойности
step	int32	числова, дискретна	0
type	str	категорийна	0
amount	float32	числова, непрекъсната	0

Колона	Тип в pandas	Категория	Брой null стойности
oldbalanceOrg	float32	числова, непрекъсната	0
newbalanceOrig	float32	числова, непрекъсната	0
nameDest	str	идентификатор (категорийна по смисъл)	0
oldbalanceDest	float32	числова, непрекъсната	0
newbalanceDest	float32	числова, непрекъсната	0
isFraud	int8	бинарна (целева)	0
isFlaggedFraud	int8	бинарна (помощна)	0

Таб. 3 Описание на типовете и броя null стойности на характеристиките

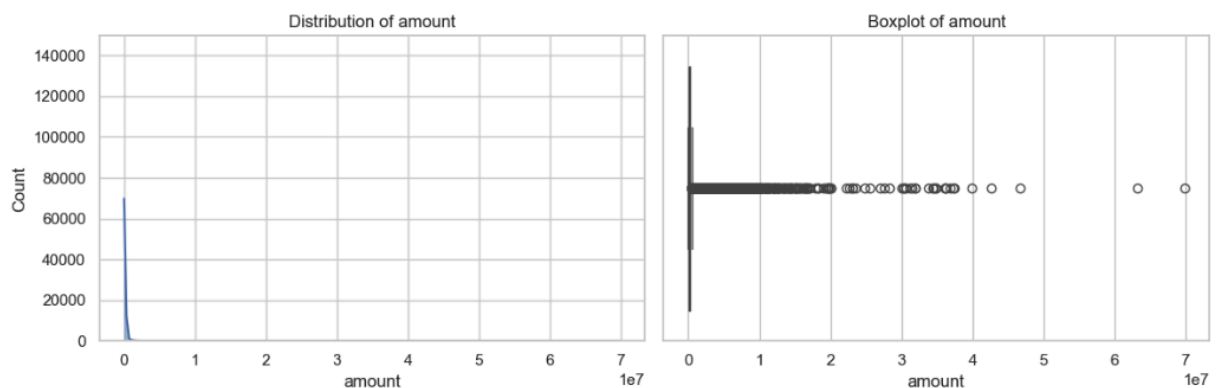
Наборът PaySim съдържа 10 колони — числови (суми и баланси), категорийна type, идентификатор nameDest и две бинарни целеви/помощни полета (isFraud, isFlaggedFraud). Всички характеристики са напълно попълнени: няма липсващи (null) стойности. Няма нужда от imputation за липсващи стойности на този етап — PaySim CSV е пълен за избраните колони.

Продължаваме с кратък анализ на всяка една от характеристиките.



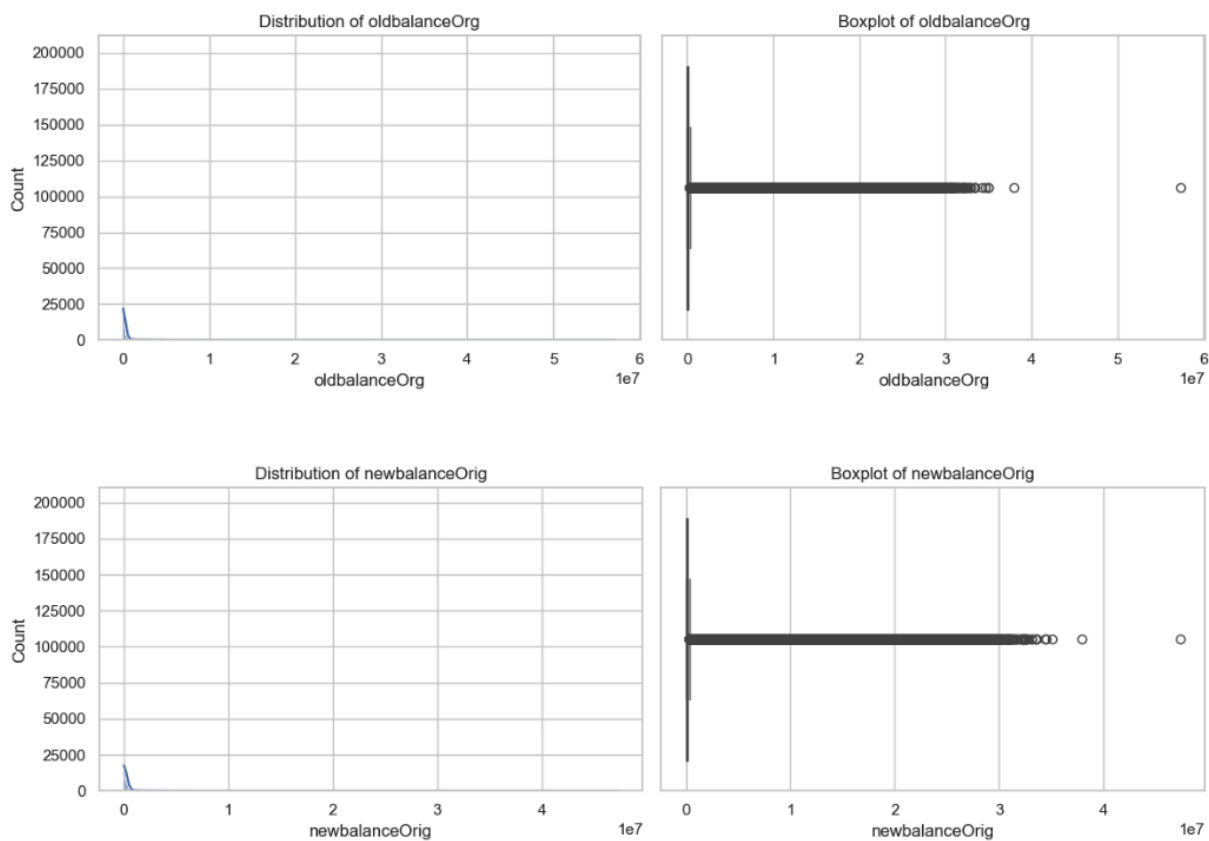
Фиг. 2 Разпределение и Boxplot диаграма на step

Разпределението е относително равномерно в интервала приблизително 1–743 (средна стойност ~ 244 , медиана ~ 240), което отразява покритие на различни часове в симулацията. Няма екстремни изолирани стойности извън whiskers на boxplot-a; данните са приблизително симетрични около средната стойност. При сравнение по isFraud разликата между класовете е слаба — времето само по себе си не е силен дискриминатор.



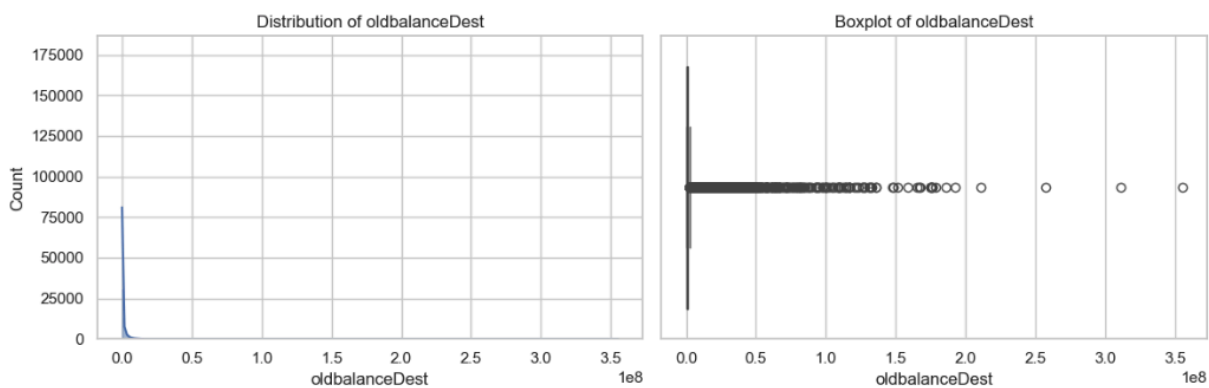
Фиг. 3 Разпределение и Boxplot диаграма на нивото на amount

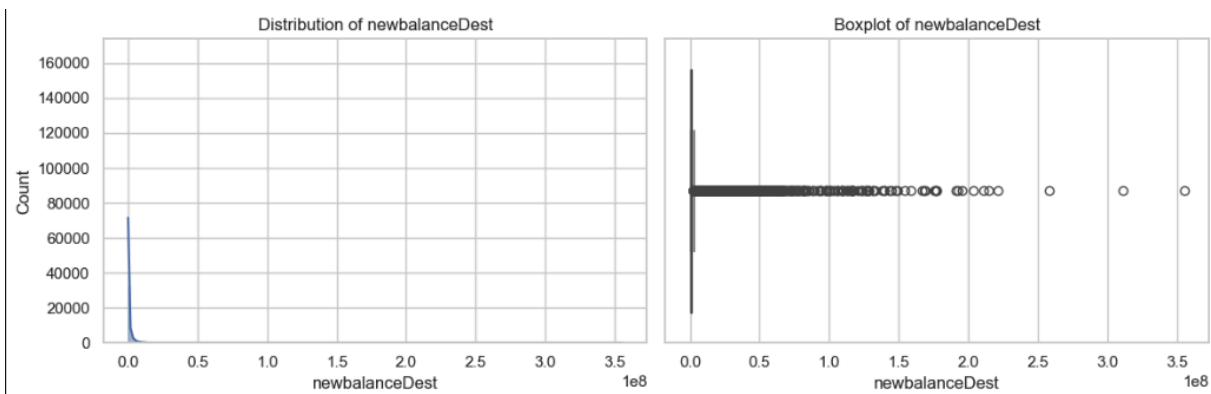
Разпределението е силно дясно отклонено: голям брой транзакции с малки и средни суми (медиана $\sim 75\,000$, 25% под $\sim 13\,400$), а опашката достига до десетки милиони. Boxplot-ът показва много outliers над горния мустак. При измами (isFraud = 1) медианата и разпределението на сумите често се отличават от легитимните — сумата е важен сигнал за моделите.



Фиг. 4 Разпределение и boxplot на балансите на изпращача

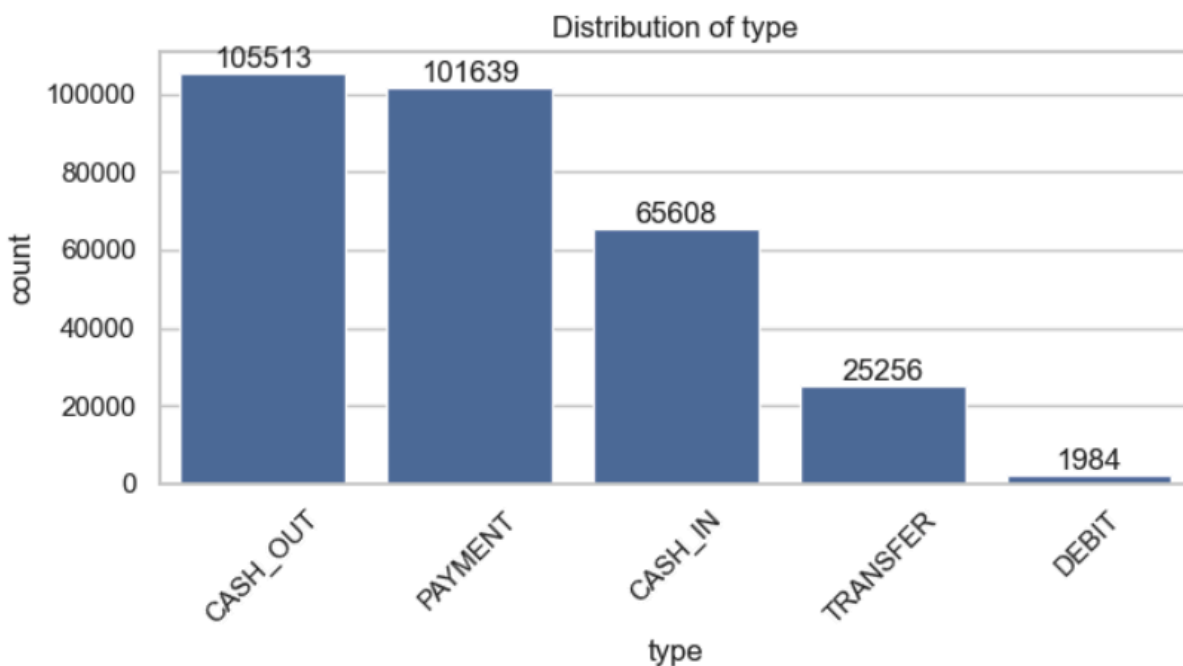
И двете характеристики имат висок дял нулеви стойности (особено newbalanceOrig — медиана 0 при 25% квантил също 0), което е типично когато изпращачът не участва активно в операцията. Останалата част е десно отклонена с екстремни стойности (до десетки милиони). Има множество outliers в boxplot-a. При isFraud разпределенията често се разминават с легитимните транзакции — несъответствия в баланса са ключови за feature engineering (errorBalanceOrig).





Фиг. 5 Разпределение и boxplot на балансите на получателя

Профилът е аналогичен на изпращача: много нули (25% квантил = 0), десен опашен разпределение и outliers. Медианите са по-високи от тези при изпращача (~133 000 / ~214 000). Boxplot-ите по isFraud показват, че измамните транзакции често имат необичайни комбинации от баланси на получателя — оттук идва и errorBalanceDest в preprocessing.



Фиг. 6 Разпределение на type

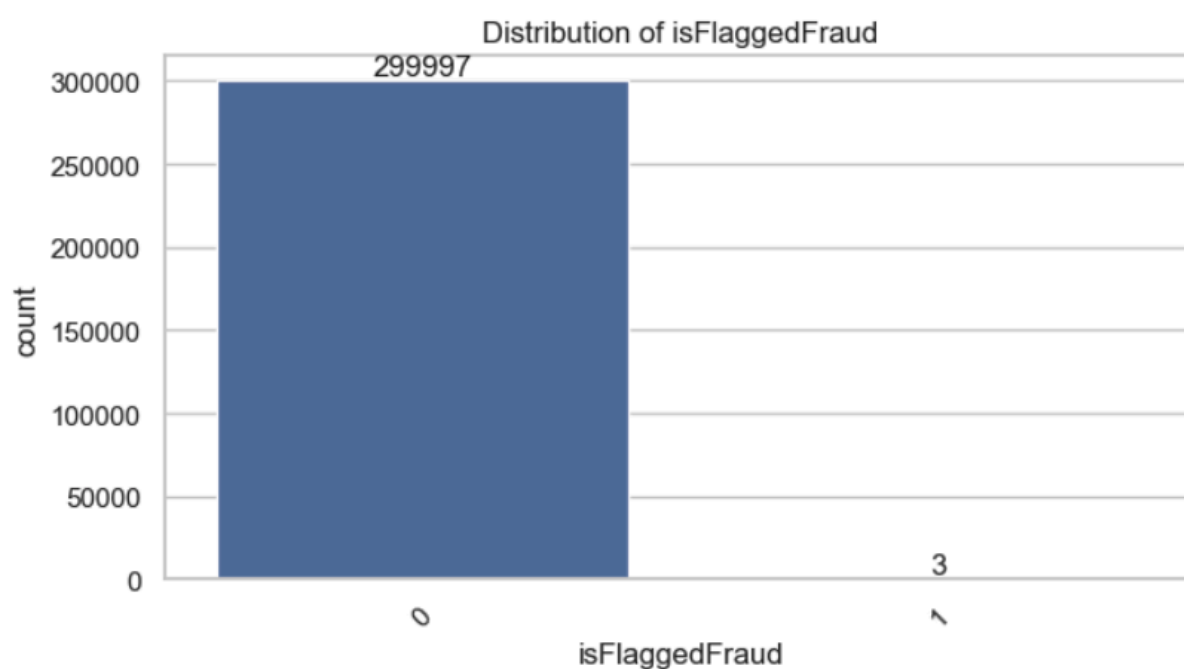
Най-често срещани са PAYMENT и TRANSFER; CASH_IN, CASH_OUT и DEBIT са по-рядки. Countplot-ът по isFraud показва, че измамните не са равномерно разпределени

по тип — TRANSFER и CASH_OUT често носят по-висок дял измами спрямо броя си транзакции. Затова type се кодира one-hot и е сред най-важните признаци в моделите.



Фиг. 7 Разпределение на isFraud

Класът е екстремно небалансиран: ~299 613 легитимни срещу 387 измамни (0,13% измами). Това оправдава метрики като PR AUC и F1, а не accuracy, и стратегиите class_weight, SMOTE и undersampling.

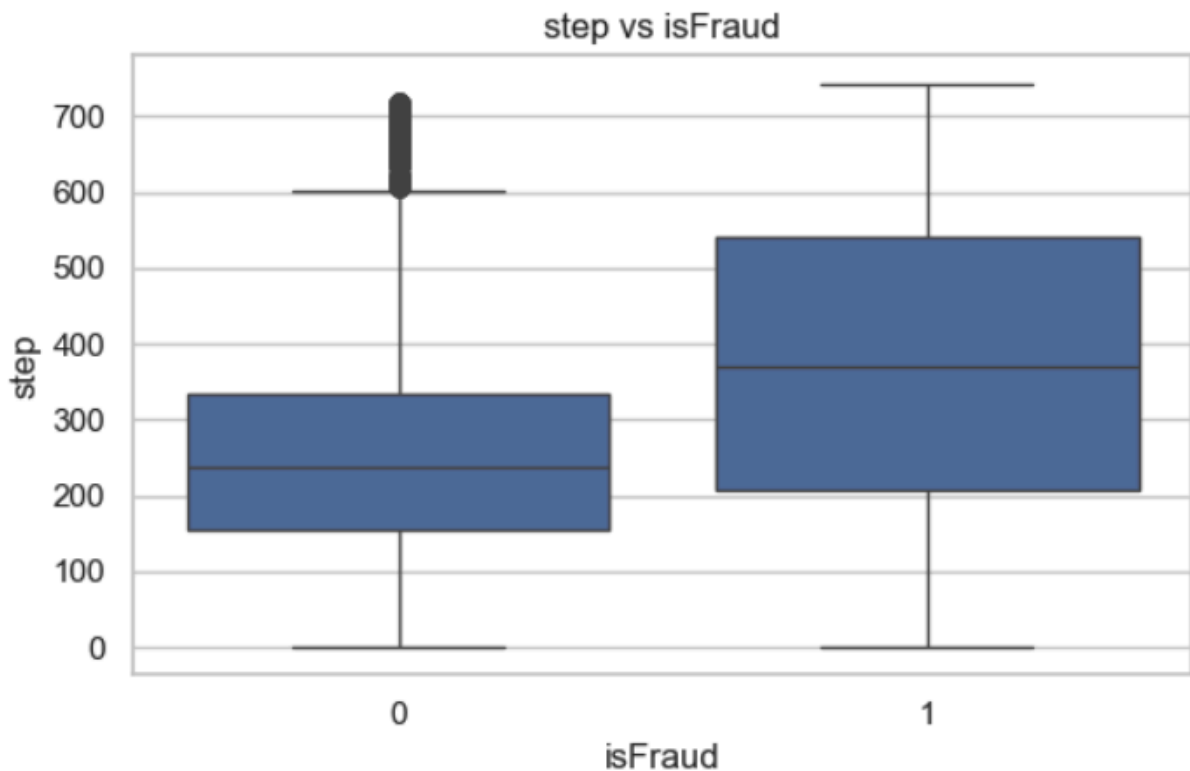


Фиг. 8 Разпределение на isFlaggedFraud

Още по-рядък от isFraud (~0,001% маркирани). Вградената rule-based система на PaySim улавя само малка част от реалните измами — показател за ограниченията на класическите правила и нуждата от ML.

Обобщение

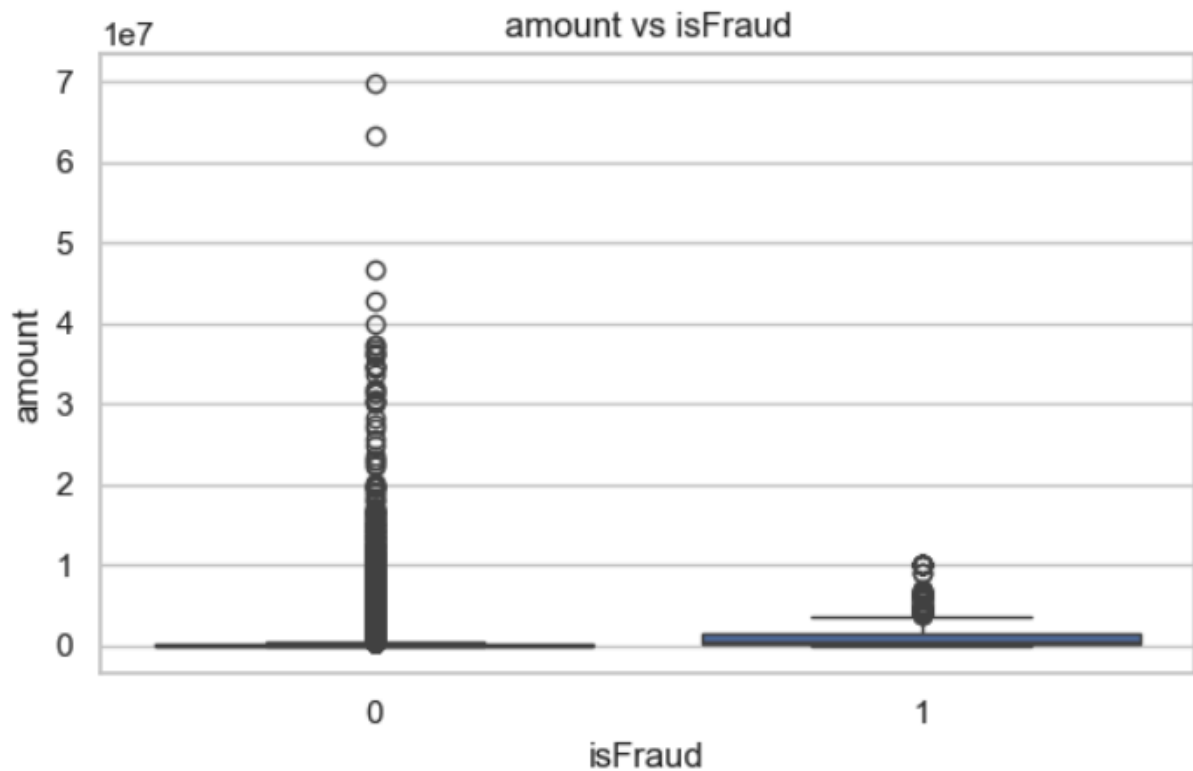
Числовите признаци са силно дясно отклонени, с много нули при балансите и outliers при сумите; категорийният type и целта isFraud са силно неравномерни. Липсват null стойности, но нулевите баланси и екстремните суми изискват внимателна обработка и метрики, подходящи за рядък положителен клас.



Фиг. 9 Boxplot на step спрямо isFraud

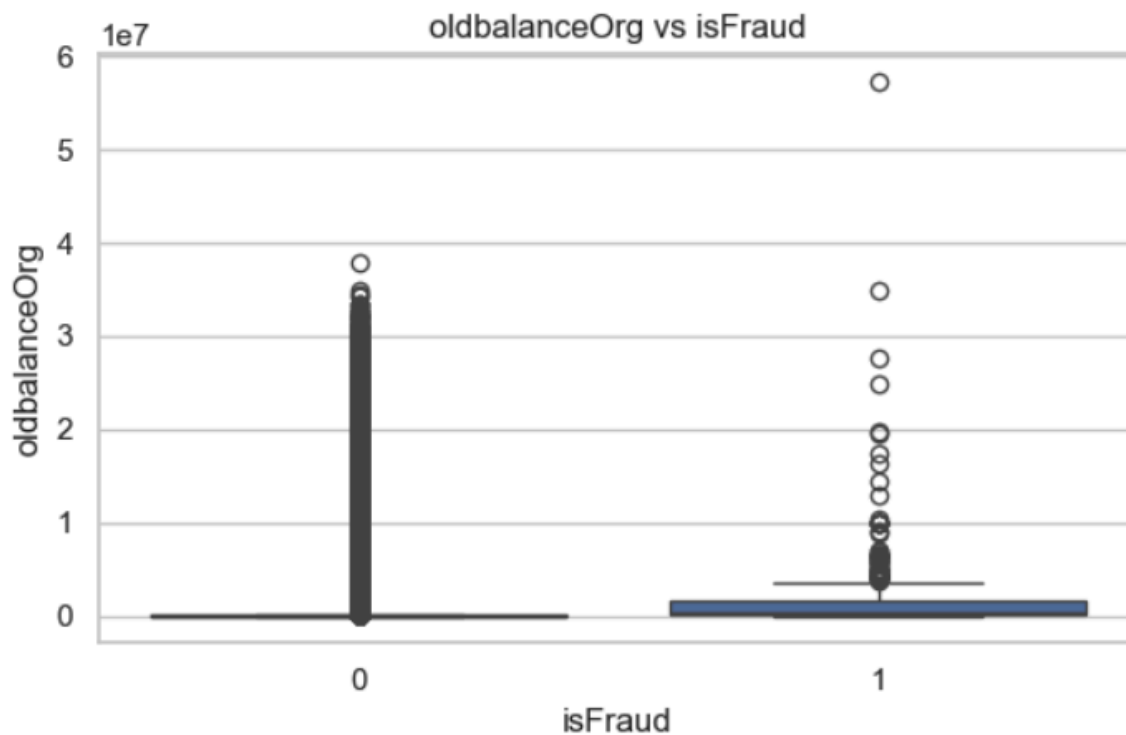
При легитимни транзакции (isFraud = 0) медианата на step е около 240 (IQR приблизително 150–340), с outliers над ~600 до ~720. При измами (isFraud = 1) медианата е значително по-висока — около 370 (IQR ~210–540), а разпределението е по-разтегнато нагоре. Извод: измамните операции в PaySim се концентрират по-късно в

симулацията (по-високи стъпки), но step сам по себе си не разделя перфектно класовете — има припокриване и при легитимни има високи стойности.



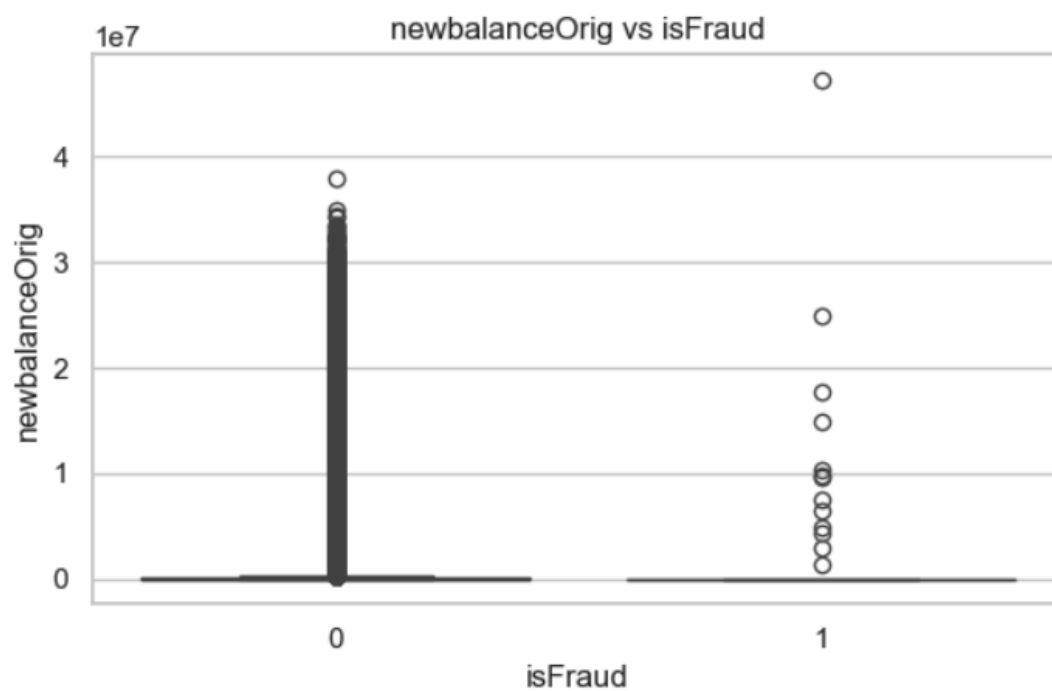
Фиг. 10 Boxplot на amount спрямо isFraud

Медианата при измами (~486 000) е многократно по-висока от при легитимните (~75 000); средната сума при измама е ~1,5M срещу ~179k. Boxplot-ът показва, че измамните транзакции са систематично по-големи. amount е един от най-силните дискриминиращи числови признаци.



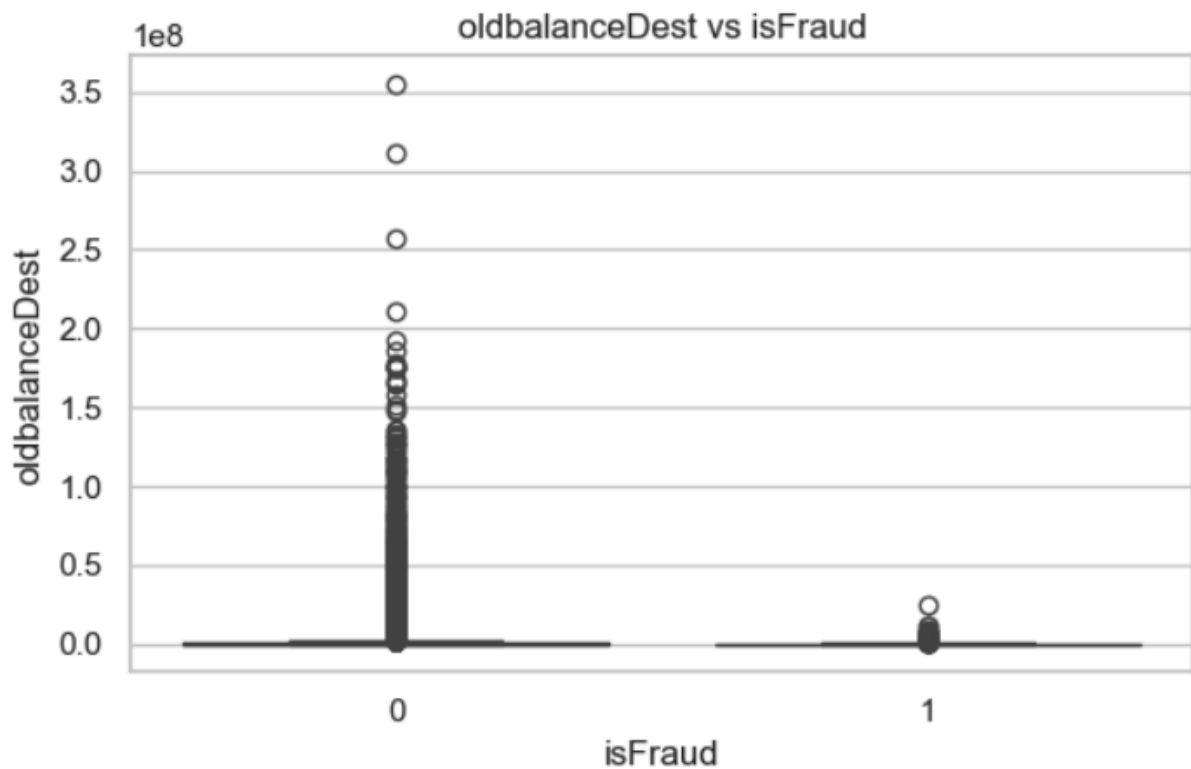
Фиг. 11 Boxplot на oldbalanceOrig спрямо isFraud

При измами медианата на баланса на изпращача преди операцията е $\sim 488\,000$ срещу $\sim 13\,600$ при легитимните; средната стойност също е по-висока. Измамите често идват от сметки с по-голям наличен баланс преди превода — логично за сценарии „източване“ на сметка.



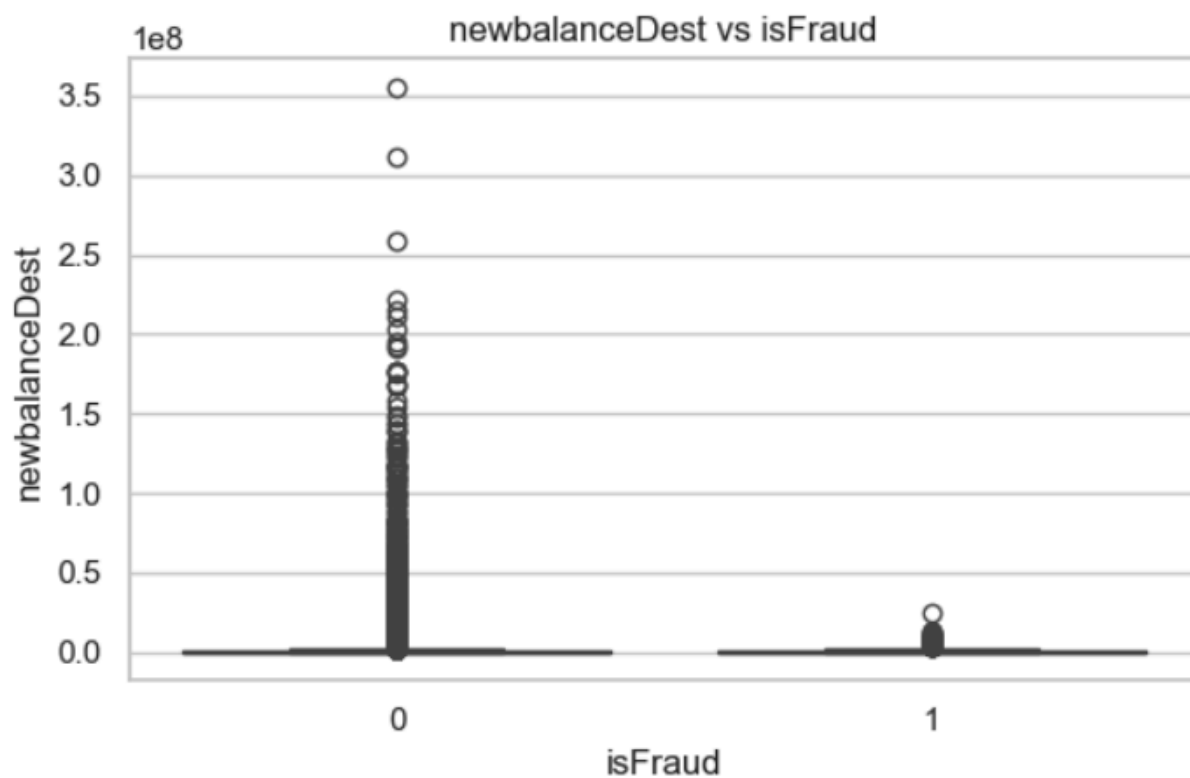
Фиг. 12 Boxplot на newbalanceOrig спрямо isFraud

И при двата класа медианата е 0 (много транзакции без активен баланс на изпращача след операцията). При измами средната стойност е по-ниска (~419k срещу ~854k), което отразява различно поведение след превода. Самият boxplot е по-слаб дискриминатор от oldbalanceOrg и amount; по-полезен е инженереният признак errorBalanceOrig.



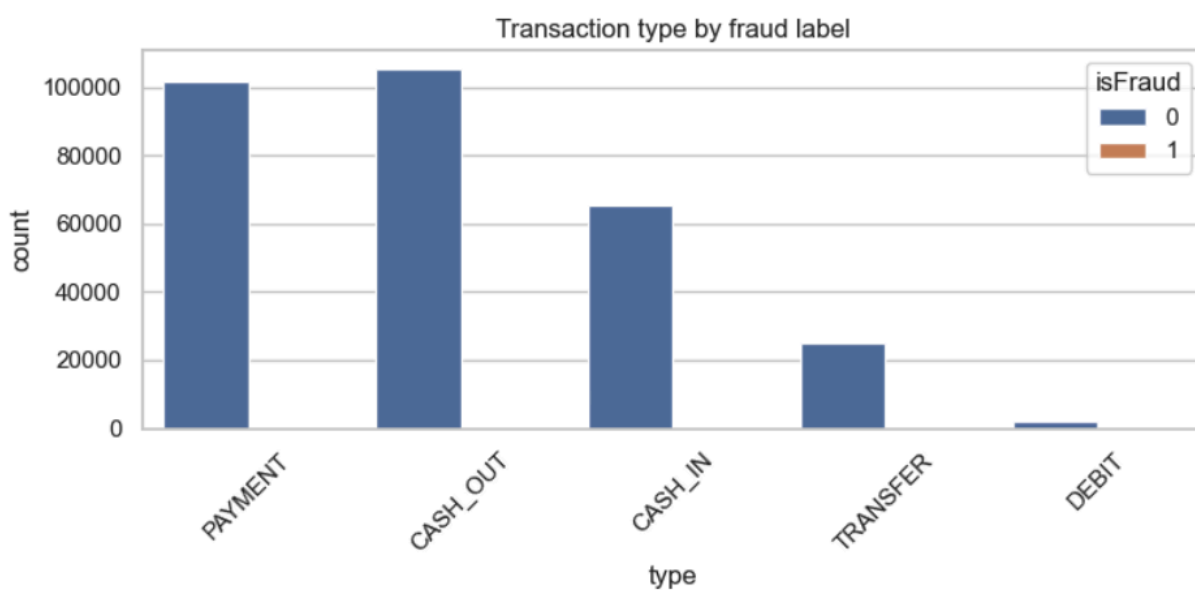
Фиг. 13 Boxplot на oldbalanceDest спрямо isFraud

При измами медианата е 0 срещу ~133 000 при легитимните — получателят често няма предишен баланс или не участва като „нормална“ сметка. Това е силен сигнал: измамните потоци често са към сметки без типичен баланс преди транзакцията.



Фиг. 14 Boxplot на newbalanceDest спрямо isFraud

Отново медиана 0 при измами срещу ~215 000 при легитимните. След операцията измамните случаи рядко оставят типичен положителен баланс у получателя. Комбинацията oldbalanceDest / newbalanceDest е ключова за откриване на аномални парични потоци.



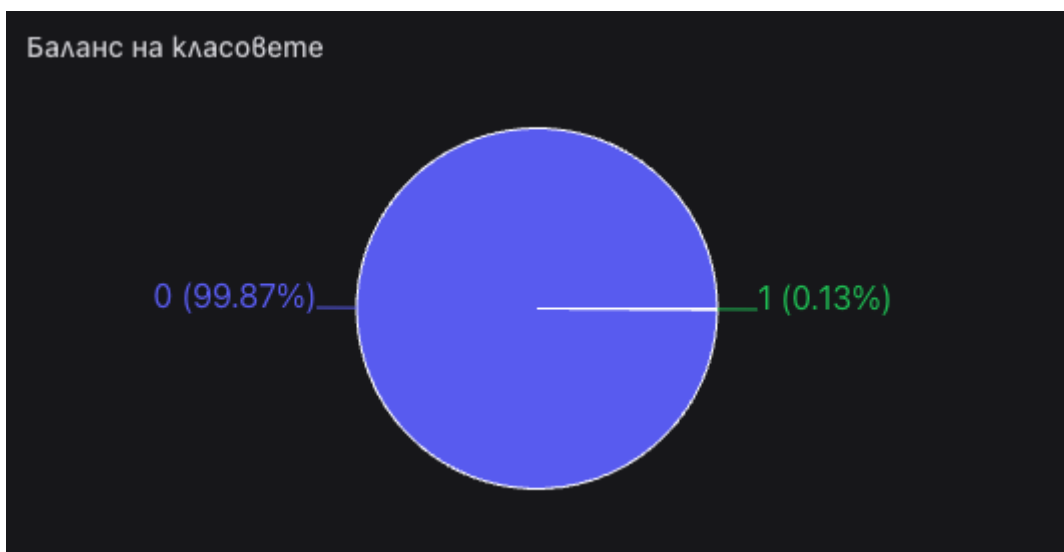
Фиг. 15 Разпределение на type по isFraud

Всички измами са в TRANSFER (~197 от 387; ~0,78% от TRANSFER) и CASH_OUT (~190; ~0,18%). PAYMENT, CASH_IN и DEBIT имат 0 измами в подизвадката. type е изключително информативен категориален признак — не всички типове носят еднакъв риск, което оправдава one-hot кодирането и включването му във всички модели.

Обобщение

Числовите признаци са силно дясно отклонени, с много нули при балансите и outliers при сумите; категоричният type и целта isFraud са силно неравномерни. Липсват null стойности, но нулевите баланси и екстремните суми изискват внимателна обработка и метрики, подходящи за рядък положителен клас. Следващата стъпка в notebook-a — feature engineering и корелации — директно използва тези наблюдения.

Анализ на данните от React дашборда

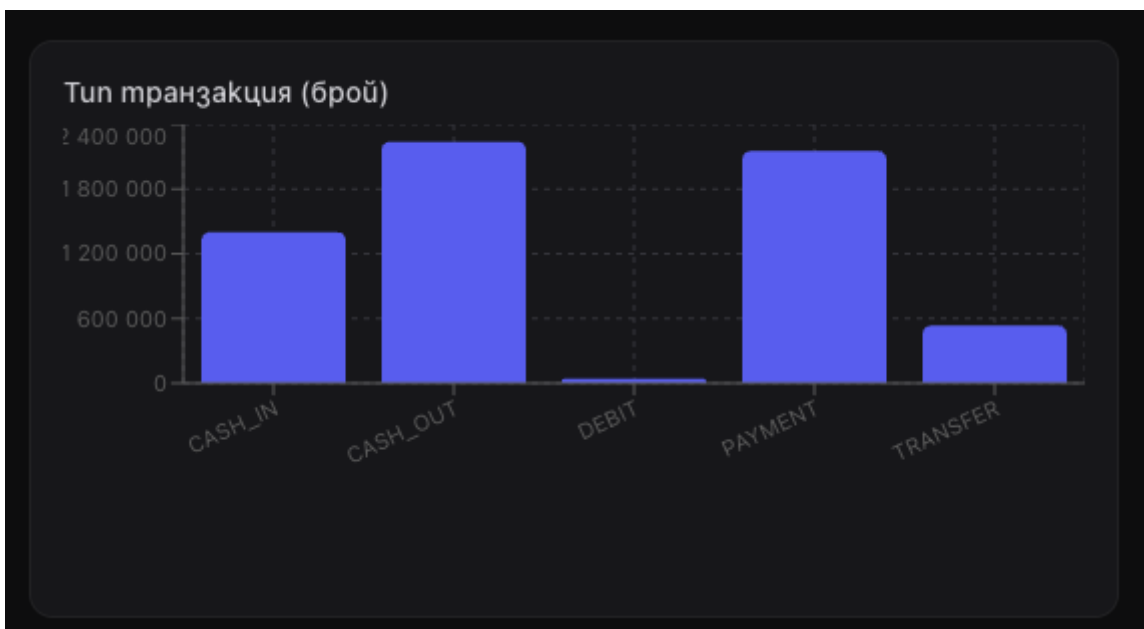


Фиг. 16 Баланс на класовете

Баланс на класовете в целия набор от данни:

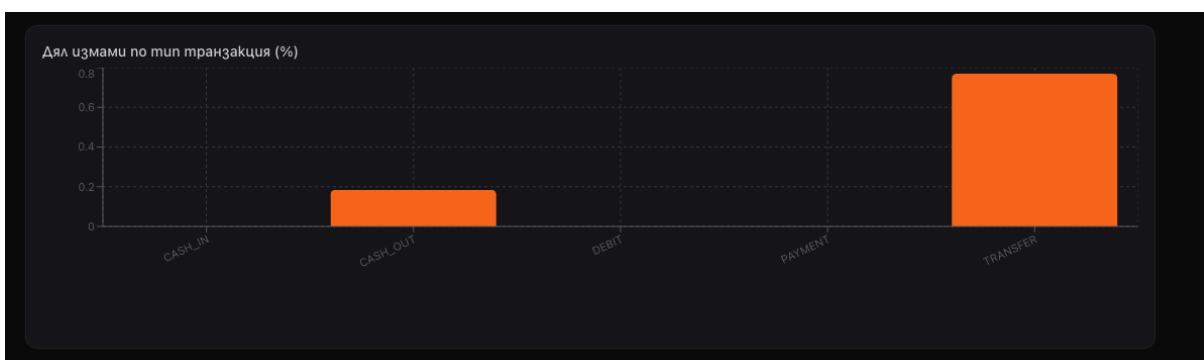
- Клас 0 (легитимни): 6 354 407 (~99,87%)
- Клас 1 (измами): 8 213 (~0,13%)

Визуализацията накратко обобщава проблема с небалансираните данни: почти целият „пай“ е легитимни транзакции. Това е полезно за читателя на дипломната работа преди секцията „Анализ“ с ML резултати.



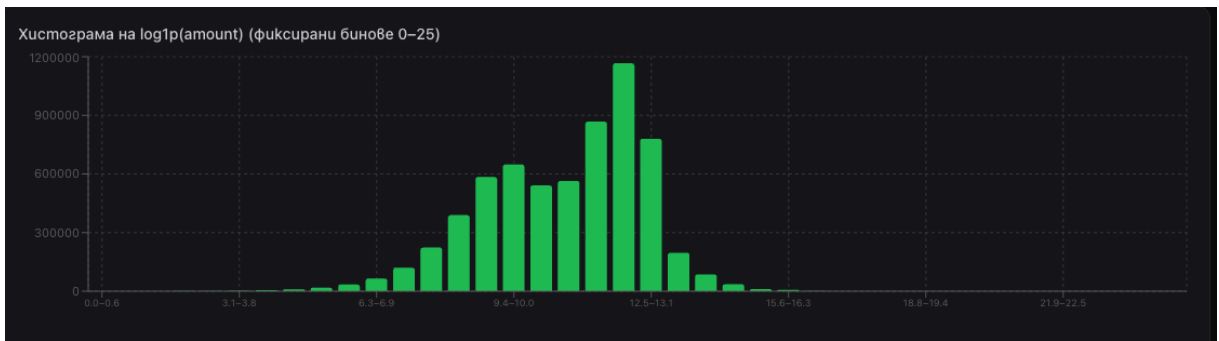
Фиг. 17 Типове транзакции

На фигурата виждаме, че най-чести са CASH_OUT (2 237 500) и PAYMENT (2 151 495) TRANSFER (532 909) и DEBIT (41 432) са по-редки по брой в целия набор от данни.



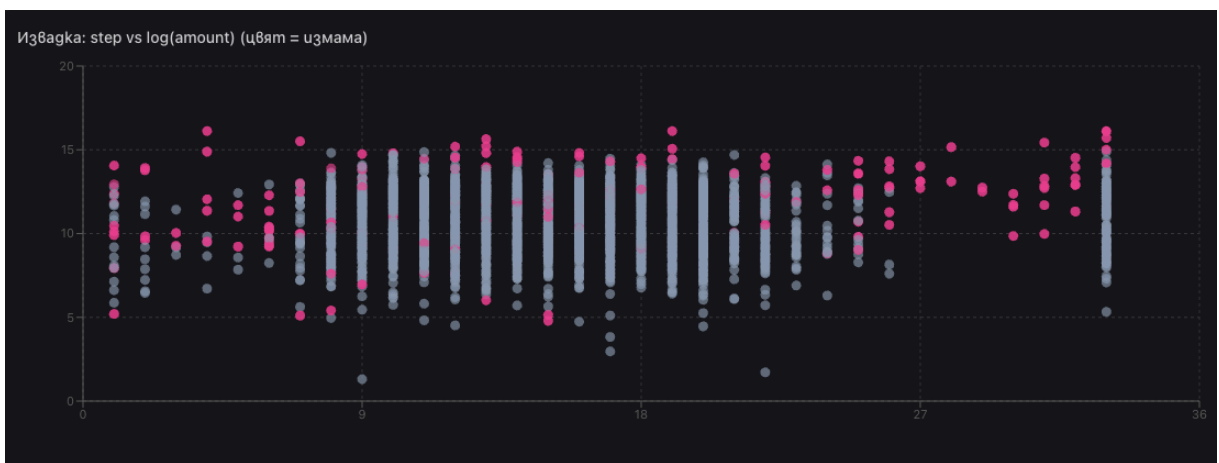
Фиг. 18 Дял измами по тип транзакция

Графиката съвпада с двувариантния анализ в notebook-a: всички измами са при TRANSFER и CASH_OUT; PAYMENT (най-голям обем) има 0 измами в пълния набор. Графиката е един от най-информативните елементи в dashboard-a — показва, че рискът е неравномерен по тип, не само по сума/баланс.



Фиг. 19 Хистограма на $\log_{1p}(\text{amount})$

Показано е разпределението на трансформираната сума $\log_{1p}(\text{amount}) = \ln(1 + \text{amount})$ в 40 фиксирани интервала между 0 и 25, изчислени върху пълния PaySim набор (6,36 млн. транзакции). Трансформацията намалява влиянието на екстремно големи суми и прави разпределението визуално сравнимо. Повечето транзакции се концентрират в интервала $\log_{1p}$ 10–13 (приблизително суми от десетки хиляди до стотици хиляди), с рядко срещани много малки и много големи стойности в опашките. Това потвърждава силната дясна асиметрия на amount в суров вид и оправдава използването на $\log$ -подобни представяния и робастни метрики при последващо моделиране на измами.



Фиг. 20 $\text{step vs } \log(\text{amount})$ (scatter, цвят = измама)

Scatter plot-ът показва стратифицирана извадка от транзакции (около 3 361 точки) от първите 600 000 записа на PaySim, с умишлено увеличен дял на измамни операции за визуална яснот. По оста Y се вижда, че измамните точки (в розово) са концентрирани при по-високи стойности на $\log_{1p}(\text{amount})$ спрямо легитимните, което подкрепя ролята на сумата като индикатор за измама. По оста X и двете групи заемат сходен диапазон на step в тази извадка, което показва, че времевият индекс сам по себе си не е достатъчен дискриминатор в показания прозорец. Графиката допълва хистограмата на

$\log_{1p}(\text{amount})$ и двувариантния анализ в notebook-a, но не заменя статистическия EDA върху обучителния набор.

Предварителна обработка на данните

След общия преглед на данните (EDA) следващата стъпка в изследването е предварителна обработка на набора PaySim. Целта е данните да бъдат приведени във вид, подходящ за обучение на алгоритми за машинно обучение: числови матрици от признаци X и целева променлива y (isFraud).

За разлика от много клинични набори, PaySim не съдържа липсващи стойности в използваните колони — проверката с `df.isnull().sum()` показва нула null стойности за всички десет полета. Затова в този проект не се прилагат техники за imputation (медиана, KNNImputer и др.); фокусът е върху инженерство на признаци, кодиране на категории, намаляване на мултиколинеарност чрез корелации и визуализация чрез анализ на главните компоненти (PCA).

Друга важна трансформация е намаляване на размерността на вектора от признаци — премахване на колони с почти идентична информация (висока корелация). Това улеснява и стабилизира обучението, особено при линейни модели (Логистична регресия, SVM), и намалява риска от нестабилни коефициенти при силно корелирани балансови полета.

Анализ на типовете характеристики

Преди кодирането структурата на данните е тази от Таб. 3. От анализа се установява, че само *type* е категориален признак в класическия смисъл; останалите полета за моделиране са числови или бинарни. Идентификаторът *nameDest* не се подава директно към модела — от него се извлича бинарен признак.

Инженерство на признаци (feature engineering)

Преди кодирането се добавят признаци, специфични за PaySim и за откриване на аномални парични потоци:

- $\text{errorBalanceOrig} = \text{oldbalanceOrg} - \text{amount} - \text{newbalanceOrig}$ — отклонение от очакваното „запазване на баланса“ при изпращача;

- $errorBalanceDest = oldbalanceDest + amount - newbalanceDest$ — същото за получателя;
- $merchantDest = 1$, ако $nameDest$ започва с "M" (търговска сметка в симулатора).

След това колоната $nameDest$ се премахва, за да се избегне директно използване на идентификатор с много уникални стойности.

Кодиране на нечисловите характеристики

Категорийният признак $type$ се преобразува с one-hot encoding (`pandas.get_dummies`, `drop_first=False`), а не с `LabelEncoder`. При fraud detection типът транзакция не е подреден номинален ред — PAYMENT, TRANSFER, CASH_OUT и др. са отделни категории с различен риск (както показва EDA: измамите са концентрирани в TRANSFER и CASH_OUT). One-hot encoding запазва тази информация без изкуствено налагане на ред.

След кодирането се получават следните колони в `df_encoded` (размер $300\,000 \times 16$):

	step	amount	oldbalanceOrig	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud	errorBalanceOrig
0	47	5661.379883	101806.0	96144.617188	0.0	0.0	0	0	0.000000
1	550	9033.589844	0.0	0.000000	0.0	0.0	0	0	-9033.589844
2	399	40710.800781	43871.0	3160.199951	0.0	0.0	0	0	-0.000732
3	255	9359.980469	0.0	0.000000	0.0	0.0	0	0	-9359.980469
4	235	3850.370117	0.0	0.000000	0.0	0.0	0	0	-3850.370117

Фиг. 21 Данни след кодирането (част 1)

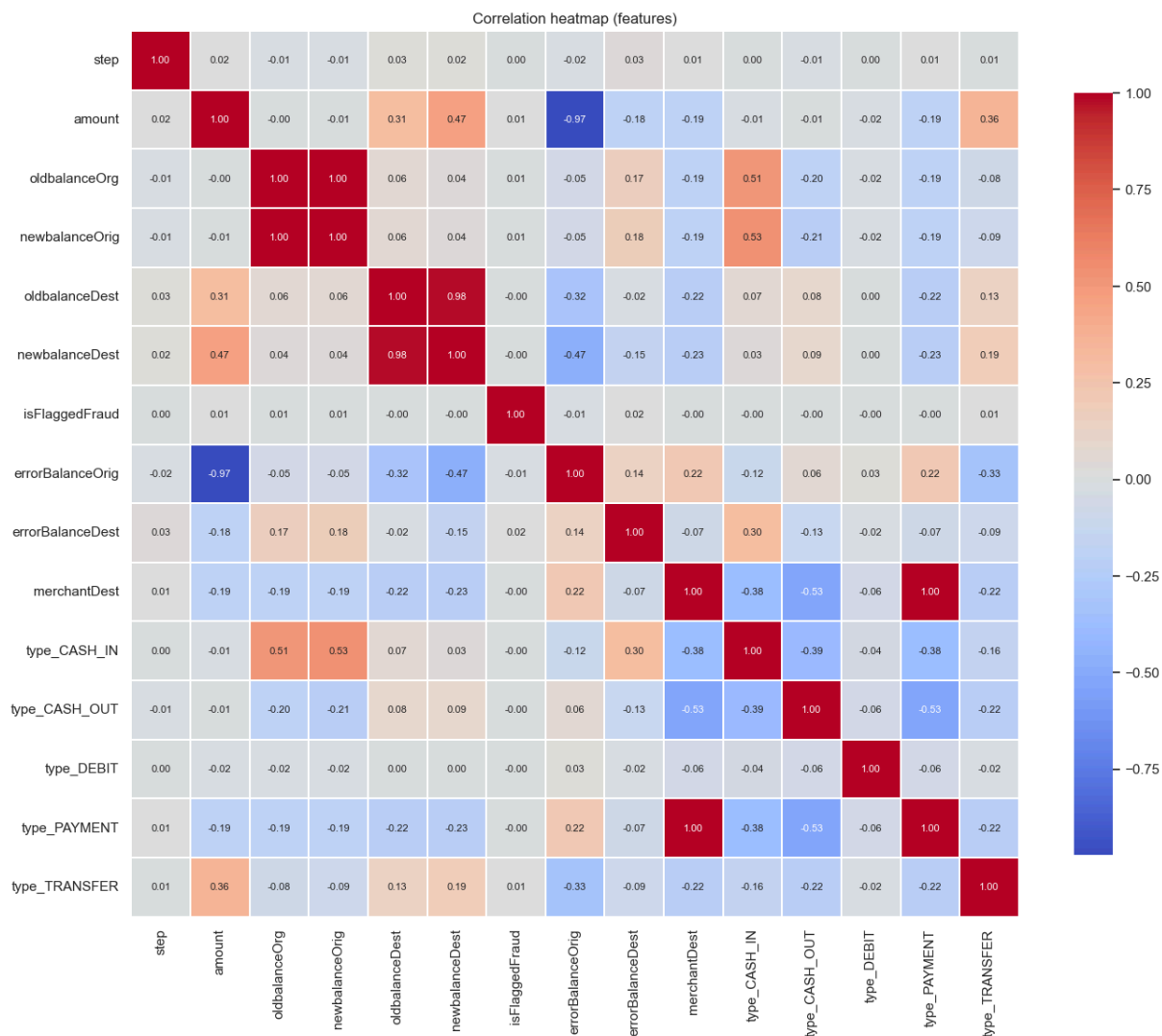
errorBalanceDest	merchantDest	type_CASH_IN	type_CASH_OUT	type_DEBIT	type_PAYMENT	type_TRANSFER
5661.379883	1	False	False	False	True	False
9033.589844	1	False	False	False	True	False
40710.800781	1	False	False	False	True	False
9359.980469	1	False	False	False	True	False
3850.370117	1	False	False	False	True	False

Фиг. 22 Данни след кодирането (част 2)

За обучение на моделите целевата променлива $isFraud$ се държи отделно; останалите 15 колони формират матрицата от признаци преди корелационния филтър.

Корелационна матрица и редуциране на размерността

Следващата стъпка е редуциране на размерността на вектора от признаци. Първият подход е матрица на корелациите (Pearson): изчислява се корелацията между всяка двойка признаци (без включване на $isFraud$ в самата матрица за този анализ) и резултатът се визуализира като топлинна карта.



Фиг. 23 Корелационна матрица на характеристиките

От корелационната матрица се вижда, че някои признаци носят сходна информация — особено между стари/нови баланси и между производните errorBalance и суровите балансови полета, както и между някои dummy колони за type. За да се намали мултиколинearността, се определя праг на корелация 0,80: от всяка двойка с $|r| > 0,80$ се премахва една от колоните (функция `reduce_dimensionality_corr`).

В случая на PaySim при този праг се премахват 4 колони:

Премахната колона	Обосновка (накратко)
newbalanceOrig	Силна корелация с oldbalanceOrig и amount

Премахната колона	Обосновка (накратко)
newbalanceDest	Силна корелация с oldbalanceDest и amount
errorBalanceOrig	Покрива се от останалите балансови признаци
type_PAYMENT	Висока корелация с други dummy за type (най-чест тип)

Таб. 4 Премахнати колони след редуциране на размерността

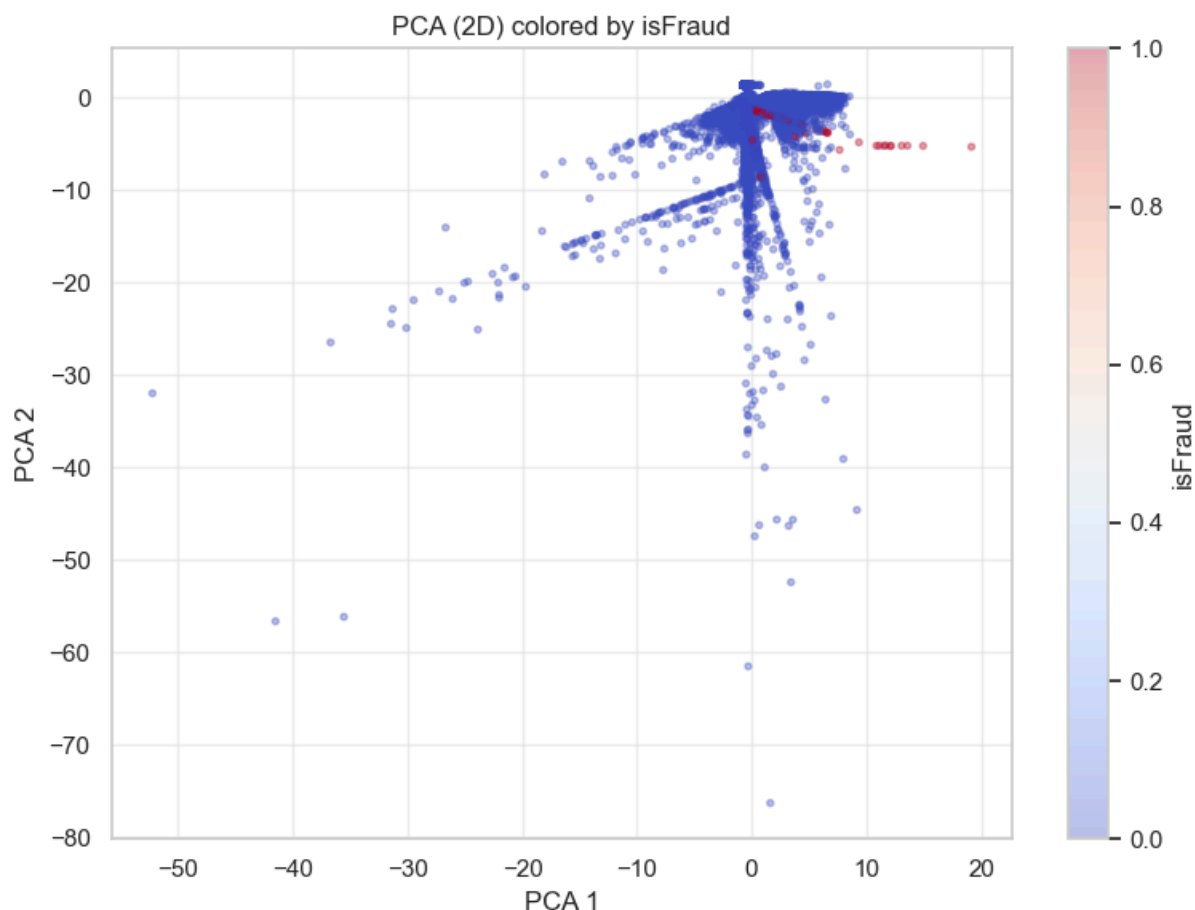
Запазени за моделиране (11 признака):

	column
0	step
1	amount
2	oldbalanceOrg
3	oldbalanceDest
4	isFlaggedFraud
5	errorBalanceDest
6	merchantDest
7	type_CASH_IN
8	type_CASH_OUT
9	type_DEBIT
10	type_TRANSFER

Така финалната матрица *df_selected* се подава към крос-валидацията и обучението на моделите в следващите секции.

Анализ на главните компоненти (PCA)

Следващата стъпка е визуализация чрез PCA върху филтрираните признаци (*df_selected*), с 2 главни компоненти и оцветяване по *isFraud*. PCA тук служи за нагледна проверка на разделимостта, а не за замяна на оригиналните признаци при обучение (моделите работят с 11-те филтрирани колони, не с PCA координатите).



Фиг. 24 2D визуализация на главните компоненти (оцветяване по isFraud)

На графиката се наблюдава частично припокриване между двете категории:

- точки за легитимни транзакции ($\text{isFraud} = 0$) — мнозинство;
- точки за измамни ($\text{isFraud} = 1$) — много малко на брой ($\sim 0,13\%$).

При PaySim няма пълно линейно разделение на двата класа в 2D PCA пространството — измамите са редки и частично се смесват с легитимните точки. Това е очаквано при екстремна небалансираност и не означава, че моделите няма да могат да откриват измами; по-скоро показва, че са нужни нелинейни модели, подходящи метрики (PR AUC, F1) и стратегии за небалансираност (секция 4), а не разчитане само на линейна разделяемост в 2D.

Въпреки припокриването, PCA потвърждава, че в данните има структура, която моделите могат да използват — в комбинация с EDA (по-високи суми и специфични типове при измами) и с feature engineering (грешки в баланса).

Обобщение

Предварителната обработка привежда PaySim от сурови транзакционни полета към 11 числови признака, готови за ML. Липсващите стойности не се попълват (няма такива в избраните колони). Категорийният type се кодира с one-hot encoding; балансовите аномалии се улавят чрез errorBalance* и merchantDest. Корелационният филтър (праг 0,80) премахва излишна информация; PCA дава визуална проверка на разделимостта при силно небалансирани класове. Следващата стъпка в pipeline-a е стратегии за работа с небалансирани данни и обучение на класификационни модели.

Класически алгоритми и начална оценка на моделите

След приключване на предварителната обработка на данните (EDA, кодиране на категориални признаци, инженеринг на признаци, корелационен анализ и подбор на признаци) се имплементират три класически модела за машинно обучение върху подготвения набор от данни. Целевата променлива е isFraud (бинарна класификация: легитимна транзакция срещу измама). Заради силната небалансираност на класовете (рядък положителен клас — измами) за всеки модел се прилага стратифицирана кръстосана валидация (StratifiedKFold) с 5 сгъвки, при която обучението и оценката се повтарят пет пъти, като съотношението между класовете се запазва във всяка сгъвка. Корекциите за небаланс (тежести на класовете, SMOTE, undersampling) се прилагат само върху обучаващата част на всяка извивка, за да се избегне изтичане на информация (data leakage) към тестовия набор.

След извършване на тренирането средната стойност на метриките от всички пет итерации се използва за обобщена оценка на производителността на модела. По-късно в експеримента се добавят и градиентни boosting модели (LightGBM, XGBoost) за сравнение, но трите алгоритъма по-долу формират базовия слой на анализа.

Random Forest

Random Forest е устойчив към шум в данните и се справя добре с високо дименсионални и хетерогенни набори от данни — характерни за транзакционни записи (суми, баланси, тип транзакция и др.). Той комбинира множество решения от различни дървета, което го прави стабилен модел с относително нисък риск от претрениране. В проекта не изисква стандартизация на признаците преди обучение.

Логистична регресия (Logistic Regression)

Логистичната регресия е линеен статистически модел за бинарна класификация и е подходяща за интерпретиране на връзката между признаците и вероятността за измама. В pipeline-а признаците се стандартизират (StandardScaler) преди обучение, тъй като моделът е чувствителен към мащаба на променливите.

Support Vector Machine (SVM)

SVM е супервизиран алгоритъм, използван най-често за класификация. Основната му идея е да намери хиперравнина (или граница в пространство от по-висока размерност),

която разделя легитимни транзакции от измамни с максимален марж. Като логистичната регресия, SVM в проекта се обучава върху скалирани признаци; за по-големи обеми данни може да се ползва линеен вариант (LinearSVC) за по-бързо обучение.

Интерпретация на грешките (аларми):

- Лъжливо положителна (FP) — моделът маркира транзакция като измама, но тя е легитимна (фалшива тревога).
- Лъжливо отрицателна (FN) — моделът пропуска реална измама (пропуснат риск).
- Истинска положителна (TP) — измама е правилно открита.
- Истинска отрицателна (TN) — легитимна транзакция е правилно приета като нормална.

Метрики за анализ на резултатите

За всеки модел и всяка стратегия за небалансираност експерименталните резултати включват показатели от матрицата на объркване (TP, TN, FP, FN) и производни метрики:

- Прецизност (Precision) - делът от предсказаните като измама транзакции, които наистина са измамни — важна при висока цена на фалшиви тревоги

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

Фиг. 25 Формула за прецизност

- Точност (Accuracy) - делът на всички правилни прогнози; при рядък положителен клас може да е подвеждащо висока, дори при слабо откриване на измами

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Всички случаи}}$$

Фиг. 26 Формула за точност

- Чувствителност/Пълнота (Sensitivity/Recall) - делът от реалните измами, които моделът успява да открие — критична за минимизиране на пропуснати измами

$$\text{Recall (Sensitivity)} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

Фиг. 27 Формула за чувствителност

- Специфичност (Specificity) - делът от легитимните транзакции, правилно класифицирани като нормални

$$\text{Specificity} = \frac{\text{True Negatives (TN)}}{\text{True Negatives (TN)} + \text{False Positives (FP)}}$$

Фиг. 28 Формула за специфичност

- F1-score - Хармонична средна между precision и recall

$$F_1 = \frac{2TP}{2TP + 2FP} = \frac{TP}{TP + FP}$$

Фиг. 29 Формула за F1-score

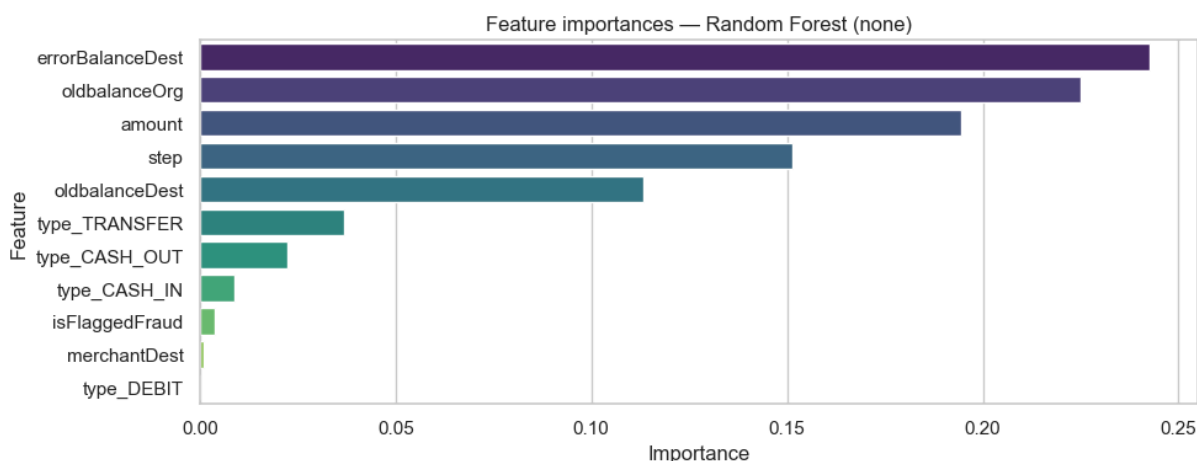
- ROC AUC (Receiver operating characteristic) - Способността на модела да ранжира измамни пред легитимни транзакции при различни прагове
- PR AUC (Area Under the Precision-Recall Curve) - По-адекватна оценка при силно небалансирани данни — фокус върху качеството върху положителния клас

Изследването анализира представянето на моделите спрямо precision, recall, specificity, accuracy, както и F1, ROC AUC и PR AUC, за да се оцени не само колко често системата „познава“ измама, а и компромисът между откриване на измами и брой фалшиви сигнали при небалансиран PaySim набор (стратифицирана подизвадка от транзакции, с характерен дял измами около 0,13%). Сравнението между Logistic Regression, Random Forest и SVM при различни стратегии за небаланс (none, class_weight, SMOTE, undersample) позволява да се определи кой подход е най-подходящ като базов референтен модел преди разширения анализ с boosting алгоритми и визуализация в React dashboard-а на проекта.

Резултати от Random Forest със стратегия none

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	50.0	59922.0	1.0	27.0	0.999533	0.980392	0.649351	0.781250	0.999983	0.958582	0.763015
1	2	52.0	59922.0	1.0	25.0	0.999567	0.981132	0.675325	0.800000	0.999983	0.959144	0.831899
2	3	50.0	59921.0	2.0	27.0	0.999517	0.961538	0.649351	0.775194	0.999967	0.939196	0.729894
3	4	63.0	59919.0	3.0	15.0	0.999700	0.954545	0.807692	0.875000	0.999950	0.973579	0.900351
4	5	56.0	59921.0	1.0	22.0	0.999617	0.982456	0.717949	0.829630	0.999983	0.966173	0.815055
5	Mean	54.2	59921.0	1.6	23.2	0.999587	0.972013	0.699933	0.812215	0.999973	0.959335	0.808043

Фиг. 30 Резултати от Random Forest



Фиг. 31 Тегло на характеристиките за Random Forest

Обобщение на производителността

Моделът улавя около 70% от измамите (recall 0,70). Това означава, че от приблизително 77 измами на всеки validation fold открива около 54, а пропуска около 23 — тоест една на всеки три измами остава незабелязана. От друга страна, когато маркира транзакция като измама, почти винаги е прав: precision е 0,97, което показва много малък брой фалшиви аларми. На fold се генерират средно само 1–2 false positive при около 60 000 легитимни транзакции, докато true negative остават стабилно около 59 921. Моделът е консервативен — предпочита да пропусне измама, отколкото да блокира нормална транзакция.

F1 score от 0,81 показва добър общ баланс между precision и recall за baseline без resampling. ROC AUC от 0,96 показва, че моделът добре разделя двата класа по вероятностен score, но при силен дисбаланс по-информативна е PR AUC (0,81) — тя отразява по-реалистично колко добре моделът работи именно върху малцинствения клас (измамите).

Feature importances — какво „вижда“ моделът

Random Forest разчита най-много на errorBalanceDest — инженерна характеристика,

която показва дали балансът на получателя след транзакцията съвпада с очакваното ($\text{oldbalanceDest} + \text{amount} - \text{newbalanceDest}$). Когато има несъответствие, това често е сигнал за измама в PaySim, затова тази променлива доминира.

След нея идват `oldbalanceOrg`, `amount` и `step` — балансът на изпращача, сумата на транзакцията и времевият индекс. Заедно с `oldbalanceDest` те формират основната част от предсказателната сила на модела: измамите се отличават по аномални суми и баланси, а не толкова по тип клиент или `merchant`.

Сред типовете транзакции най-важни са `TRANSFER` и `CASH_OUT`, което съвпада с типичните fraud сценарии в dataset-a. `CASH_IN` и `DEBIT` имат малък принос.

`isFlaggedFraud` и `merchantDest` практически не участват в решението — `isFlaggedFraud` в PaySim рядко съвпада с реалната измама, а дали получателят е `merchant` почти не променя предсказанието.

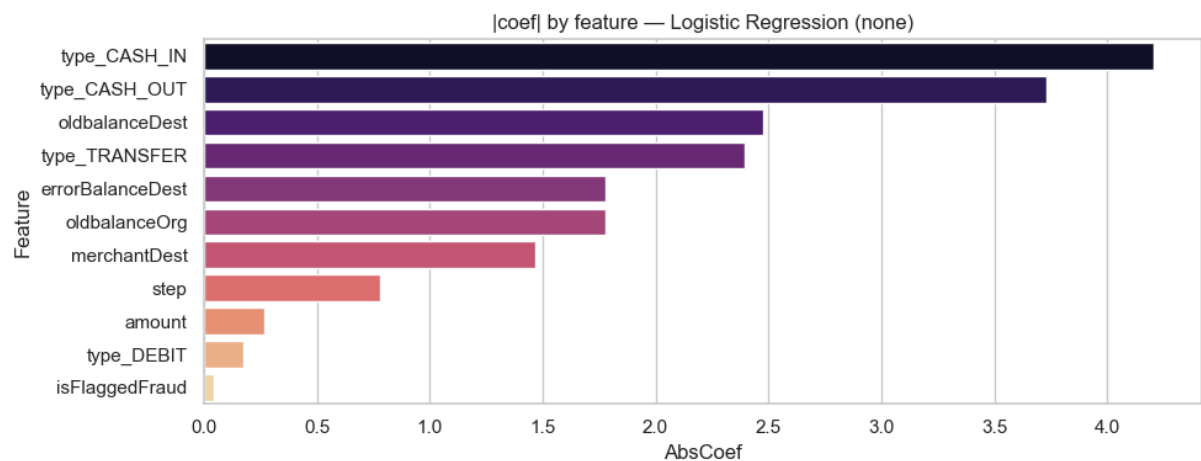
Обобщение

Random Forest без корекция за дисбаланс дава надежден, консервативен детектор на измами: почти не блокира легитимните транзакции (FP ~1–2 на fold), но пропуска ~30% от измамите. Моделът разчита главно на балансови аномалии (`errorBalanceDest`) и сумата/баланси, което е логично за PaySim. Резултатите са добри за baseline, но изборът на стратегия зависи от бизнес приоритета.

Резултати от Логистична регресия със стратегия none

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	21.0	59923.0	0.0	56.0	0.999067	1.000000	0.272727	0.428571	1.000000	0.934611	0.445525
1	2	18.0	59922.0	1.0	59.0	0.999000	0.947368	0.233766	0.375000	0.999983	0.928604	0.336134
2	3	17.0	59919.0	4.0	60.0	0.998933	0.809524	0.220779	0.346939	0.999933	0.952950	0.360563
3	4	26.0	59918.0	4.0	52.0	0.999067	0.866667	0.333333	0.481481	0.999933	0.950815	0.469073
4	5	14.0	59920.0	2.0	64.0	0.998900	0.875000	0.179487	0.297872	0.999967	0.931911	0.369662
5	Mean	19.2	59920.4	2.2	58.2	0.998993	0.899712	0.248019	0.385973	0.999963	0.939778	0.396191

Фиг. 32 Резултати от Логистична регресия



Фиг. 33 Тегло на характеристиките за Логистична регресия

Обобщение на производителността

Logistic Regression без корекция за дисбаланс е значително по-слаб от Random Forest при същата задача. Асигура остава висока (99,9%), но това отново е следствие от дисбаланса, не от реална сила на модела.

Средно моделът открива само около 25% от измамите (recall 0,25) — от ~77 измами на fold улавя ~19 и пропуска ~58. Това е много по-лош резултат от RF (~70% recall). Precision е висока (0,90): когато моделът каже „измама“, в 9 от 10 случая е прав. False positive остават малко (~2 на fold), но цената е твърде много пропуснати измами.

F1 е само 0,39 — слаб баланс между precision и recall. ROC AUC (0,94) изглежда добре, но PR AUC е едва 0,40 — при дисбаланс това показва, че моделът слабо работи върху малцинствения клас, въпреки че вероятностните score-ове разделят класовете относително добре.

Моделът е изключително консервативен — почти не маркира измами, за да избегне грешки. За откриване на измами това е проблем: 3 от 4 измами остават незабелязани, което го прави неподходящ като самостоятелен детектор без допълнителни корекции.

Feature importances — какво „вижда“ моделът

Logistic Regression гледа на данните различно от Random Forest. Тук най-силно влияят типовете транзакции — type_CASH_IN, type_CASH_OUT и type_TRANSFER имат най-големи коефициенти. Линейният модел разчита главно на категориалните разлики между типовете, а не на сложни балансови взаимодействия.

След тях идват oldbalanceDest, errorBalanceDest и oldbalanceOrg — балансовите променливи все още имат значение, но по-малко отколкото при RF. merchantDest и step участват умерено.

Интересно е, че amount има почти нулево влияние — за разлика от RF, където сумата е сред топ 3 features. isFlaggedFraud отново е без значение. type_DEBIT е най-слаб сред типовете (вероятно референтна категория при one-hot encoding).

Обобщение

Logistic Regression (none) е подходящ само като референция/baseline — показва, че линейният модел и default threshold не са достатъчни за PaySim откриване на измами. За практическа употреба tree-based модели (RF, XGBoost) или tuning на threshold/class weights са по-подходящи.

Резултати от Support Vector Machine (SVM) със стратегия none

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	16.0	59923.0	0.0	61.0	0.998983	1.000000	0.207792	0.344086	1.000000	0.950683	0.482676
1	2	15.0	59920.0	3.0	62.0	0.998917	0.833333	0.194805	0.315789	0.999950	0.954160	0.404924
2	3	15.0	59921.0	2.0	62.0	0.998933	0.882353	0.194805	0.319149	0.999967	0.971420	0.386425
3	4	18.0	59918.0	4.0	60.0	0.998933	0.818182	0.230769	0.360000	0.999933	0.977021	0.483239
4	5	11.0	59922.0	0.0	67.0	0.998883	1.000000	0.141026	0.247191	1.000000	0.952100	0.440770
5	Mean	15.0	59920.8	1.8	62.4	0.998930	0.906774	0.193839	0.317243	0.999970	0.961077	0.439607

Фиг. 34 Резултати от SVM

Обобщение на производителността

Моделът е много консервативен и улавя само около 19% от измамите (recall 0,19). На fold от ~77 реални измами открива ~15 и пропуска ~62 — над 4 от 5 измами остават незабелязани. Това е още по-слабо от Logistic Regression (~25% recall) и далеч от Random Forest (~70%).

Precision е висока (0,91): когато моделът маркира измама, почти винаги е прав. False positive са малко (~1–2 на fold), specificity е почти перфектна (99,997%). Accuracy от 99,89% отново е подвеждаща — следствие от дисбаланса, не от добра детекция на измами.

F1 е 0,32 — слаб баланс. ROC AUC (0,96) изглежда добре, но PR AUC е само 0,44 — при дисбаланс това показва слаба работа върху малцинствения клас. Разликата между ROC и PR AUC е типична: моделът рангира добре, но при default threshold 0.5 почти не предсказва измами.

Между fold-овете recall варира от 14% до 23%, PR AUC от 0,39 до 0,48 — умерена нестабилност при малко положителни примери на fold.

Какво „вижда“ моделът

В notebook-а няма графика за feature importances за SVM (за разлика от RF и Logistic Regression). LinearSVC е линеен модел с StandardScaler, така че логиката му е близка до LR: разчита на линейни комбинации от features, не на сложни взаимодействия.

При сходни метрики с LR (recall ~19–25%, precision ~90%) е вероятно SVM да ползва сходни сигнали — тип транзакция, баланси и errorBalanceDest — но по-агресивна margin-based граница води до още по-малко positive predictions и по-нисък recall.

SMOTE стратегия за работа с небалансирани данни

SMOTE (Synthetic Minority Over-sampling Technique) е техника за работа с небалансирани класове. В PaySim измамите са ~387 от 300 000 (~0,13%) — моделът вижда прекалено малко fraud примери и склонен е да предсказва почти винаги „не измама“, както при strategy=none.

SMOTE не копира съществуващи fraud записи, а генерира синтетични — за всеки реален fraud пример намира близки съседи сред малцинството и създава нови точки по линии между тях в множеството от характеристики. Целта е моделът да „види“ повече разнообразие от зависимости между имамите по време на обучение.

В notebook-а SMOTE се прилага само върху train частта на всеки fold, след StratifiedKfold split. Validation/test fold остава непроменен — реално разпределение (~77 измами на fold). Това е важно: ако SMOTE се приложи преди split, информация от test „изтича“ в train (data leakage). Правилният ред е: split → SMOTE само на X_train, y_train → scaling (ако е нужно) → fit → оценка на оригиналния test fold.

Конкретно за PaySim: в train fold има ~310 реални измами и ~240 000 legit транзакции. SMOTE по default балансира класовете — създава стотици хиляди синтетични fraud примери, докато двата класа станат приблизително равни. Моделът се обучава на „изкуствено балансиран“ train set, но се оценява на реално небалансиран test set.

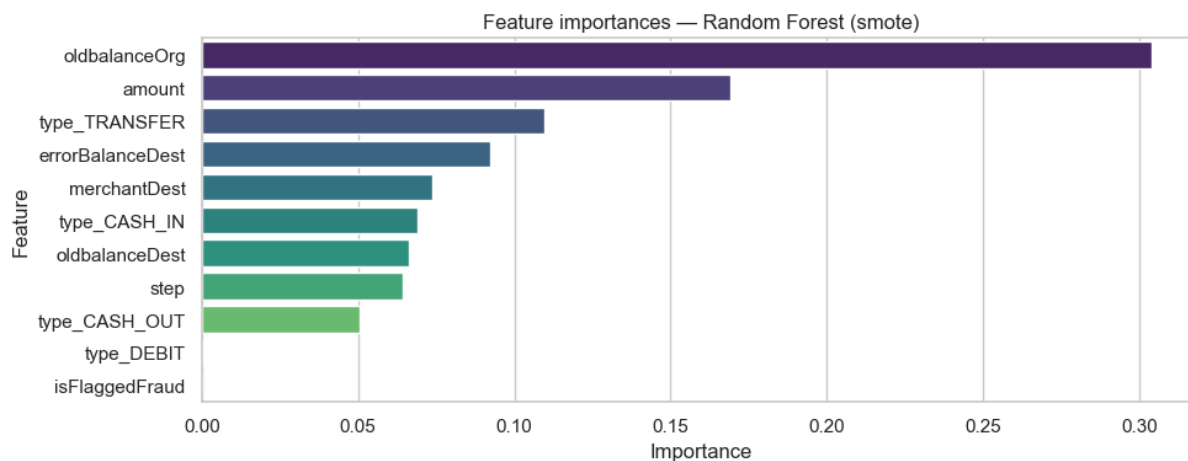
Очакван ефект спрямо none: по-висок recall (повече открити измами), но по-нисък precision (повече false positive), защото моделът е по-агресивен към fraud класа. F1 и PR AUC могат да се подобрят или влошат — зависи от trade-off-a. SMOTE не променя hyperparameters на модела; за sklearn моделите при smote не се ползват class_weight=balanced (тежестите са изключени, защото resampling вече „балансира“ данните).

Ограничения: синтетичните примери може да не отразяват реални fraud patterns; при много features или шум SMOTE може да създаде нереалистични комбинации. Затова резултатите от SMOTE трябва да се интерпретират с PR AUC и confusion matrix, не само с accuracy.

Резултати от Random Forest със стратегия smote

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	62.0	59848.0	75.0	15.0	0.998500	0.452555	0.805195	0.579439	0.998748	0.998791	0.800306
1	2	62.0	59853.0	70.0	15.0	0.998583	0.469697	0.805195	0.593301	0.998832	0.984809	0.792456
2	3	61.0	59864.0	59.0	16.0	0.998750	0.508333	0.792208	0.619289	0.999015	0.992096	0.796003
3	4	71.0	59853.0	69.0	7.0	0.998733	0.507143	0.910256	0.651376	0.998849	0.992299	0.880279
4	5	67.0	59847.0	75.0	11.0	0.998567	0.471831	0.858974	0.609091	0.998748	0.998602	0.835513
5	Mean	64.6	59853.0	69.6	12.8	0.998627	0.481912	0.834366	0.610499	0.998839	0.993319	0.820911

Фиг. 35 Резултати от Random Forest



Фиг. 36 Тегло на характеристиките за Random Forest

Обобщение на производителността

С SMOTE Random Forest става значително по-агресивен в откриването на измами. Recall скача от ~70% (none) на ~83% — от ~77 измами на fold улавя ~65 и пропуска само ~13. Това е ясно подобрене: почти 5 от 6 измами се откриват, докато при baseline се пропускаха ~30%.

Цената е precision — пада от ~97% на ~48%. Когато моделът каже „измама“, е прав в половината от случаите. False positive нарастват от ~1–2 на fold до ~70 — все още малък дял от ~60 000 legit транзакции (~0,12%), но много повече от baseline.

F1 пада от 0,81 на 0,61 — по-ниският precision тежи повече от по-високия recall. Accuracy остава ~99,86% и отново е подвеждаща. PR AUC (0,82) е леко по-добра от none (0,81), ROC AUC (0,99) също — моделът рангира малко по-добре, но при default threshold trade-off-ът е ясен.

Между fold-овете recall варира от ~79% до ~91%, precision от ~45% до ~51% — умерена стабилност. Specificity остава висока (99,88%), но не перфектна като при none.

Обобщено за производителността: SMOTE при RF е подходящ, когато пропуснатата измама е по-скъпа от фалшива аларма. Ако приоритетът е минимум false positive, none остава по-добър избор.

Feature importances — какво „вижда“ моделът

С SMOTE важността на features се променя спрямо none. Най-важна е oldbalanceOrg (~0,30), следвана от amount (~0,17) и type_TRANSFER (~0,11). errorBalanceDest, доминиращ при none (~0,24), тук пада на ~0,09 — моделът разчита по-малко на балансовите несъответствия при получателя.

По-видими стават merchantDest и type_CASH_IN — SMOTE добавя синтетични fraud примери и моделът „научава“ допълнителни patterns. isFlaggedFraud и type_DEBIT отново са без значение.

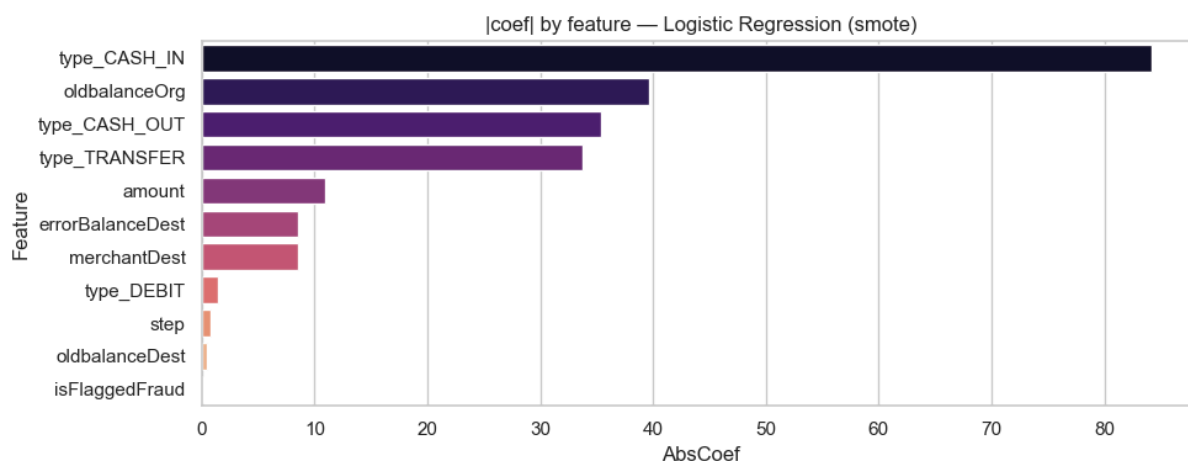
Обобщение

При SMOTE RF „вижда“ измама предимно през баланса на изпращача, сумата и TRANSFER транзакции — по-различен профил от conservative baseline, където errorBalanceDest беше водещ.

Резултати от Логистична регресия със стратегия smote

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	68.0	56325.0	3598.0	9.0	0.939883	0.018549	0.883117	0.036334	0.939956	0.976334	0.320073
1	2	72.0	56262.0	3661.0	5.0	0.938900	0.019287	0.935065	0.037795	0.938905	0.980293	0.219502
2	3	75.0	56224.0	3699.0	2.0	0.938317	0.019873	0.974026	0.038951	0.938271	0.983218	0.202542
3	4	77.0	56105.0	3817.0	1.0	0.936367	0.019774	0.987179	0.038771	0.936301	0.988530	0.251812
4	5	69.0	56380.0	3542.0	9.0	0.940817	0.019108	0.884615	0.037409	0.940890	0.973793	0.272501
5	Mean	72.2	56259.2	3663.4	5.2	0.938857	0.019318	0.932801	0.037852	0.938864	0.980434	0.253286

Фиг. 37 Резултати от Логистична регресия



Фиг. 38 Тегло на характеристиките за Логистична регресия

Обобщение на производителността

SMOTE при Logistic Regression дава екстреман trade-off — почти противоположен на none. Recall скача от ~25% на ~93%: от ~77 измами на fold улавя ~72 и пропуска само ~5. Моделът „хваща“ почти всички измами.

Цената е precision ~1,9% — от ~3 735 положителни предсказания на fold (~72 TP + ~3 663 FP) само ~72 са реални измами. 98 от 100 аларми са фалшиви. False positive нарастват от ~2 на fold до ~3 663 — ~6% от legit транзакциите на fold се маркират погрешно.

F1 е 0,038 — practically нулев. Accuracy пада от ~99,9% на ~93,9% — един от малкото случаи, в които accuracy реално отразява проблема. PR AUC (0,25) е по-лоша от none (0,40), въпреки recall 93% — при дисбаланс precision е критична и моделът я проваля. ROC AUC (0,98) остава висок, но е подвеждащ: score-овете рангират добре, но default threshold 0.5 води до масово positive predictions.

Specificity пада от ~99,99% на ~93,9% — моделът „крие“ fraud твърде агресивно и блокира хиляди legit транзакции.

Feature importances — какво „вижда“ моделът

С SMOTE коефициентите се раздуват и променят профила спрямо none. type_CASH_IN доминира (~84) — далеч над останалите; при none беше ~4.2. Следват oldbalanceOrg (~40), type_CASH_OUT (~35) и type_TRANSFER (~33). amount (~11) и errorBalanceDest (~8) имат по-малко влияние отколкото при RF.

isFlaggedFraud и oldbalanceDest отново са без значение. Линеиният модел след SMOTE се „залепва“ за тип транзакция — особено CASH_IN — и баланса на изпращача, вероятно защото синтетичните fraud примери усилват тези patterns.

Обобщение

Моделът разчита на груби линейни правила по тип транзакция, което при балансиран train води до масово маркиране на цели категории като подозрителни.

Резултати от Support Vector Machine (SVM) със стратегия smote

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	67.0	56195.0	3728.0	10.0	0.937700	0.017655	0.870130	0.034607	0.937787	0.972421	0.345904
1	2	70.0	56132.0	3791.0	7.0	0.936700	0.018130	0.909091	0.035551	0.936735	0.976716	0.240095
2	3	73.0	56189.0	3734.0	4.0	0.937700	0.019175	0.948052	0.037590	0.937687	0.981780	0.257886
3	4	76.0	56158.0	3764.0	2.0	0.937233	0.019792	0.974359	0.038795	0.937185	0.988207	0.259213
4	5	68.0	56242.0	3680.0	10.0	0.938500	0.018143	0.871795	0.035546	0.938587	0.971004	0.324187
5	Mean	70.8	56183.2	3739.4	6.6	0.937567	0.018579	0.914685	0.036418	0.937596	0.978026	0.285457

Фиг. 39 Резултати от SVM

Обобщение на производителността

SMOTE при LinearSVC дава същия екстремн профил като при Logistic Regression. Recall е ~91% — от ~77 измами на fold улавя ~71 и пропуска само ~7. Почти всички реални измами се откриват.

Precision е ~1,9% — от ~3 810 положителни предсказания на fold (~71 TP + ~3 739 FP) само ~71 са истински измами. Почти всяка аларма е фалшива. False positive нарастват от ~2 на fold (none) до ~3 739 — около 6% от legit транзакциите се маркират погрешно.

F1 е 0,036, PR AUC 0,29 — по-лоши от none (F1 0,32, PR AUC 0,44), въпреки recall ~91%. ROC AUC (0,98) отново е misleading. Accuracy пада от ~99,9% на ~93,8%, specificity от ~99,99% на ~93,8%.

Между fold-овете recall е 87–97%, precision стабилно ~1,8–2,0% — поведението е последователно, но непрактично.

Обобщено: SVM + SMOTE с default threshold е неподходящ за production — висок recall при цена от хиляди фалшиви аларми на fold.

Какво „вижда“ моделът

В notebook-а няма графика за feature importances за SVM. LinearSVC е линеен модел с StandardScaler, така че логиката му е близка до LR + SMOTE: след балансиране на train моделът става агресивен и маркира масово транзакции като изми — вероятно по тип транзакция и балансови features, както при Logistic Regression.

Сравнение и обобщение при strategy=smote

SMOTE повишава recall при трите модела, но не по един и същ начин. Random Forest намира най-добър баланс: recall ~83%, precision ~48%, F1 0,61, ~70 false positive и ~13 пропуснати измами на fold. PR AUC 0,82 показва, че моделът работи приемливо върху малцинствения клас — SMOTE тук дава реален, контролируем trade-off.

Logistic Regression и SVM реагират по сходен и краен начин: recall ~93% и ~91%, но precision ~2% — от хиляди аларми на fold (~3 700+) само ~70–72 са истински измами. F1 е ~0,04, PR AUC ~0,25–0,29. Линеините модели след SMOTE стават твърде агресивни при default threshold 0.5 и генерират масови фалшиви аларми (~6% от legit транзакциите на fold).

SMOTE е полезна стратегия само при Random Forest в този случай — повече открити измами при приемлива цена. При LR и SVM резултатите са непрактични.

Class weight стратегия за работа с небалансирани данни

class_weight е подход, при който данните не се променят — няма синтетични примери (като SMOTE) и няма изтриване на записи (като undersampling). Вместо това моделът се „наказва“ по-силно при грешка върху малцинствения клас (измамите).

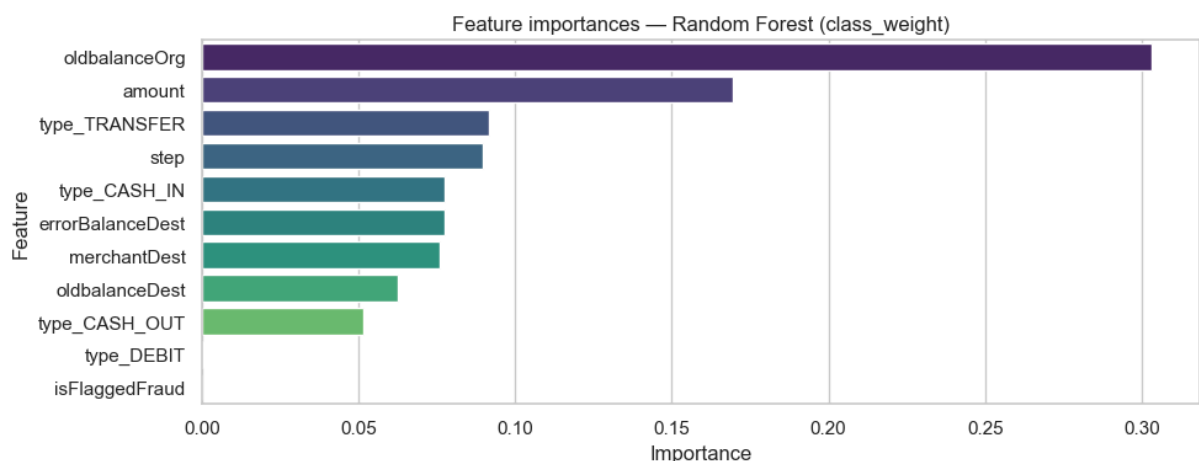
При sklearn моделите (Logistic Regression, Random Forest, SVM) в notebook-а се ползва class_weight="balanced" — тежестите се изчисляват обратно пропорционално на честотата на класа. В PaySim с ~387 измами и ~299 613 legit транзакции fraud примерите получават много по-голяма тежест (~774×) при обучение. Моделът „обръща повече внимание“ на пропуснатата измама, отколкото на грешка при legit транзакция.

За разлика от SMOTE, train set остава оригинален по размер и състав — няма изтичане на данни от resampling, но моделът е по-чувствителен към fraud класа чрез loss/penalty. Очаква се по-висок recall от none, без да се създават хиляди синтетични примери.

Резултати от Random Forest със стратегия class_weight

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	49.0	59922.0	1.0	28.0	0.999517	0.980000	0.636364	0.771654	0.999983	0.966092	0.776048
1	2	52.0	59923.0	0.0	25.0	0.999583	1.000000	0.675325	0.806202	1.000000	0.953151	0.831820
2	3	50.0	59921.0	2.0	27.0	0.999517	0.961538	0.649351	0.775194	0.999967	0.972462	0.768601
3	4	62.0	59921.0	1.0	16.0	0.999717	0.984127	0.794872	0.879433	0.999983	0.973512	0.886560
4	5	55.0	59919.0	3.0	23.0	0.999567	0.948276	0.705128	0.808824	0.999950	0.960403	0.817892
5	Mean	53.6	59921.2	1.4	23.8	0.999580	0.974788	0.692208	0.808261	0.999977	0.965124	0.816184

Фиг. 40 Резултати от Random Forest



Фиг. 41 Тегло на характеристиките за Random forest

Обобщение на производителността

При Random Forest class_weight почти не променя резултатите спрямо none. Recall е ~69% (53–62 TP, ~24 FN на fold), precision ~97%, F1 0,81, PR AUC 0,82. False positive остават ~1–2 на fold — моделът остава консервативен и почти не генерира фалшиви аларми.

Разликите с none са минимални: recall 69% vs 70%, precision 97,5% vs 97,2%, F1 0,808 vs 0,812. class_weight="balanced" практически не влияе на Random Forest при PaySim — tree-based моделът вече улавя fraud patterns без изрични тежести.

Feature importances — какво „вижда“ моделът

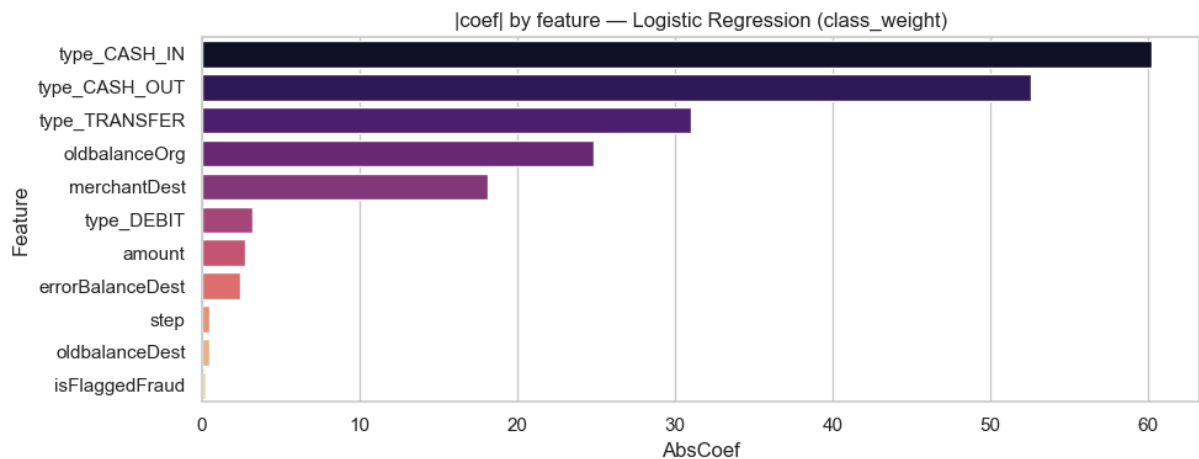
Профилът е по-близо до SMOTE, отколкото до none. Най-важни са oldbalanceOrg (~0,30), amount (~0,17) и type_TRANSFER (~0,09). errorBalanceDest, доминиращ при none (~0,24), тук е ~0,08. isFlaggedFraud и type_DEBIT отново са без значение.

За Random Forest class_weight не добавя стойност спрямо none — сходни метрики при минимални FP. За по-висок recall без екстремни FP smote е по-подходящ (~83% recall, ~70 FP). При RF class_weight може да се пропусне — none дава същия резултат.

Резултати от Логистична регресия със стратегия class_weight

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	68.0	55942.0	3981.0	9.0	0.933500	0.016794	0.883117	0.032962	0.933565	0.976134	0.288959
1	2	71.0	55772.0	4151.0	6.0	0.930717	0.016817	0.922078	0.033031	0.930728	0.979515	0.195356
2	3	75.0	55877.0	4046.0	2.0	0.932533	0.018199	0.974026	0.035731	0.932480	0.982342	0.180695
3	4	77.0	55583.0	4339.0	1.0	0.927667	0.017437	0.987179	0.034268	0.927589	0.988283	0.200966
4	5	69.0	56036.0	3886.0	9.0	0.935083	0.017446	0.884615	0.034218	0.935149	0.974408	0.270365
5	Mean	72.0	55842.0	4080.6	5.4	0.931900	0.017339	0.930203	0.034042	0.931902	0.980136	0.227268

Фиг. 42 Резултати от Логистична регресия



Фиг. 43 Тегло на характеристиките за Логистична регресия

Обобщение на производителността

При Logistic Regression class_weight коренно променя поведението — по същия начин като SMOTE, не като при Random Forest. Recall скача от ~25% (none) на ~93%: от ~77 измами на fold улавя ~72 и пропуска само ~5. Почти всички реални измами се откриват.

Цената е precision ~1,7% — от ~4 152 аларми на fold (~72 TP + ~4 081 FP) само ~72 са истински измами. Почти всяка аларма е фалшива. False positive нарастват от ~2 на fold до ~4 081 — ~6,8% от legit транзакциите се маркират погрешно.

F1 е 0,034, PR AUC 0,23 — по-лоши от none (F1 0,39, PR AUC 0,40). ROC AUC (0,98) отново е misleading. Accuracy пада от ~99,9% на ~93,2%.

Feature importances — какво „вижда“ моделът

Доминират type_CASH_IN (~60), type_CASH_OUT (~53) и type_TRANSFER (~31), следвани от oldbalanceOrg (~25) и merchantDest (~18). amount и errorBalanceDest имат малко влияние; isFlaggedFraud е без значение.

Моделът се „залепва“ за тип транзакция — при висока тежест на fraud класа линейните правила по категории водят до масово маркиране на цели групи като подозрителни.

За Logistic Regression class_weight и SMOTE дават почти идентичен резултат — висок recall, неприемливо нисък precision. За практическа употреба none е по-разумен (precision ~90%), или е нужен threshold tuning след обучение с тежести.

Резултати от Support Vector Machine (SVM) със стратегия class_weight

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	68.0	55978.0	3945.0	9.0	0.934100	0.016945	0.883117	0.033252	0.934166	0.974288	0.306345
1	2	70.0	55823.0	4100.0	7.0	0.931550	0.016787	0.909091	0.032964	0.931579	0.977626	0.194935
2	3	74.0	55968.0	3955.0	3.0	0.934033	0.018367	0.961039	0.036045	0.933999	0.981556	0.223904
3	4	77.0	55846.0	4076.0	1.0	0.932050	0.018541	0.987179	0.036398	0.931978	0.988279	0.215892
4	5	69.0	56108.0	3814.0	9.0	0.936283	0.017770	0.884615	0.034840	0.936351	0.973512	0.302523
5	Mean	71.6	55944.6	3978.0	5.8	0.933603	0.017682	0.925008	0.034700	0.933614	0.979052	0.248720

Фиг. 44 Резултати от SVM

Обобщение на производителността

При SVM class_weight променя поведението по същия начин като при Logistic Regression — от твърде консервативен модел към твърде агресивен. Recall скача от ~19% (none) на ~93%: от ~77 измами на fold улавя ~72 и пропуска само ~6. Почти всички реални измами се откриват.

Precision е ~1,8% — от ~4 050 аларми на fold (~72 TP + ~3 978 FP) само ~72 са истински измами. Почти всяка аларма е фалшива. False positive нарастват от ~2 на fold до ~3 978 — ~6,6% от legit транзакциите се маркират погрешно.

F1 е 0,035, PR AUC 0,25 — по-лоши от none (F1 0,32, PR AUC 0,44). ROC AUC (0,98) отново е misleading. Accuracy пада от ~99,9% на ~93,4%, specificity от ~99,99% на ~93,4%.

SVM + class_weight с default threshold е непрактичен за production — същият профил като LR + class_weight и LR/SVM + SMOTE.

Какво „вижда“ моделът

В notebook-a няма графика за feature importances за SVM. LinearSVC с class_weight="balanced" е линеен модел — при ~774× по-голяма тежест на fraud класа decision boundary се измества агресивно и моделът маркира масово транзакции като измами, вероятно по тип транзакция и балансови features, както при Logistic Regression.

За SVM `class_weight`, SMOTE и `class_weight` при LR дават почти еднакъв резултат — висок `recall`, нисък `precision`. За практическа употреба `none` е по-разумен (`precision` ~91%), или е нужен `threshold tuning`. Единственият модел, при който `class_weight` работи разумно, е Random Forest — там тежестите почти не променят резултата спрямо `none`.

Undersample стратегия за работа с небалансирани данни

Undersampling намалява дисбаланса като премахва записи от преобладаващия клас (легитимните транзакции), вместо да добавя синтетични fraud примери (SMOTE) или да променя тежестите (`class_weight`).

В notebook-а се ползва RandomUnderSampler от imbalanced-learn — по default балансира класовете, като намалява majority класа до броя на minority. В PaySim train fold има ~310 измами и ~240 000 legit транзакции; след undersampling остават ~310 от всеки клас — ~239 000 legit записа се изхвърлят от train set.

Както при SMOTE, undersampling се прилага само върху train частта на всеки fold, след split — test fold остава с реалното разпределение (~77 измами на ~60 000 транзакции), за да няма data leakage.

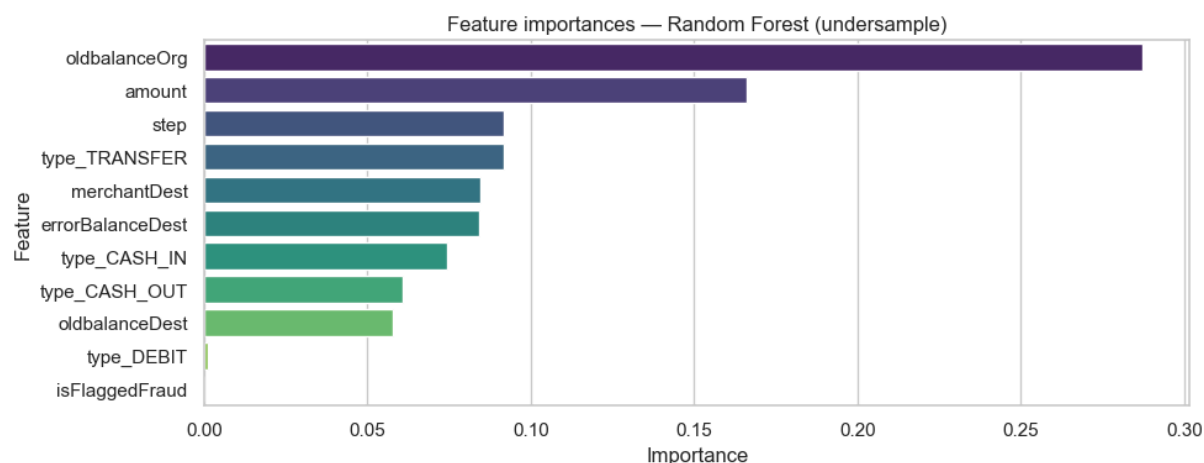
Очакван ефект: моделът се обучава на малък, балансиран train set и става агресивен към fraud — обикновено много висок `recall`, но много нисък `precision` и много false positive, защото majority patterns са изхвърлени и моделът „не вижда“ достатъчно legit транзакции по време на обучение.

Губи се огромна част от информацията от legit класа — при PaySim това често води до модел, който маркира прекалено много транзакции като изми при оценка на реално небалансиран test set.

Резултати от Random Forest със стратегия undersample

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	75.0	57139.0	2784.0	2.0	0.953567	0.026233	0.974026	0.051090	0.953540	0.995391	0.707581
1	2	73.0	57172.0	2751.0	4.0	0.954083	0.025850	0.948052	0.050327	0.954091	0.993227	0.545976
2	3	74.0	57408.0	2515.0	3.0	0.958033	0.028582	0.961039	0.055514	0.958029	0.993400	0.684528
3	4	78.0	57150.0	2772.0	0.0	0.953800	0.027368	1.000000	0.053279	0.953740	0.996905	0.681558
4	5	75.0	57063.0	2859.0	3.0	0.952300	0.025562	0.961538	0.049801	0.952288	0.995565	0.692376
5	Mean	75.0	57186.4	2736.2	2.4	0.954357	0.026719	0.968931	0.052002	0.954338	0.994898	0.662403

Фиг. 45 Резултати от Random Forest



Фиг. 46 Тегло на характеристиките за Random forest

Обобщение на производителността

Undersampling при Random Forest дава най-висок recall от всички стратегии — ~97%: от ~77 измами на fold улавя ~75 и пропуска само ~2. Почти никоя измама не остава незабелязана.

Цената е precision ~2,7% — от ~2 811 аларми на fold (~75 TP + ~2 736 FP) само ~75 са реални измами. 97 от 100 аларми са фалшиви. False positive нарастват до ~2 736 на fold — ~4,6% от legit транзакциите — повече от SMOTE (~70 FP), но по-малко от LR/SVM с class_weight или SMOTE (~3 700+ FP).

F1 е 0,052 — най-нисък от всички RF конфигурации. PR AUC 0,66 е по-лоша от none (0,81) и SMOTE (0,82), въпреки recall 97%. ROC AUC (0,99) отново е misleading. Accuracy пада на ~95,4%.

RF + undersample улавя почти всички измами, но генерира хиляди фалшиви аларми — по-лош trade-off от SMOTE при същия модел.

Feature importances — какво „вижда“ моделът

Най-важни са oldbalanceOrg (~0,29), amount (~0,17), step (~0,09) и type_TRANSFER (~0,09). errorBalanceDest (~0,05–0,08) е по-слаб от при none. isFlaggedFraud и type_DEBIT са без значение.

Моделът се обучава на ~620 балансиран запис (310 fraud + 310 legit) и „вижда“ измама през баланс на изпращача и сума — но без достатъчно legit patterns от train set маркира масово транзакции като подозрителни на test.

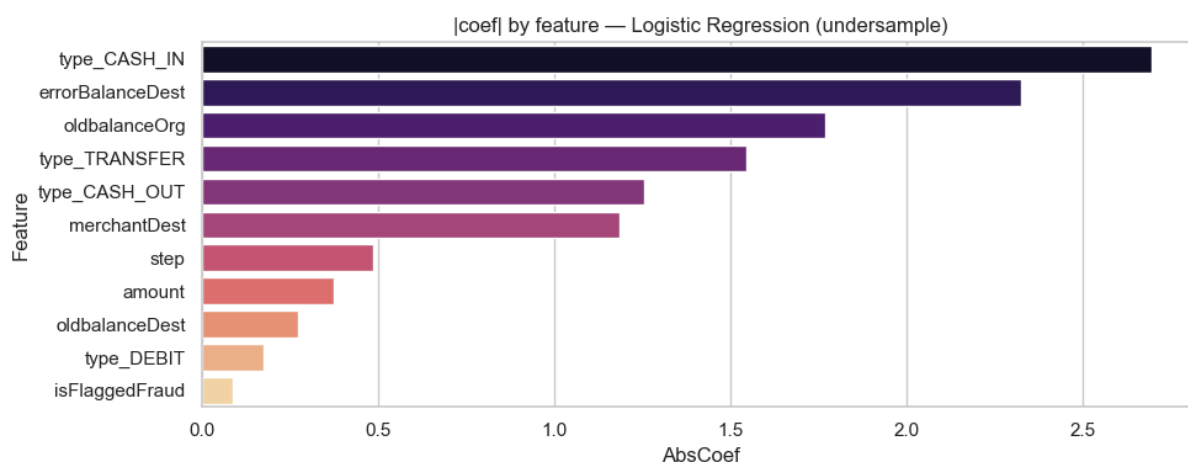
За Random Forest undersample е най-лошият trade-off — recall е най-висок, но precision и F1 са катастрофални. SMOTE е по-добър избор за по-висок recall (~83% при ~48%

precision). none остава най-добър за минимум false positive (F1 0,81, precision 97%).

Резултати от Логистична регресия със стратегия undersample

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	65.0	47252.0	12671.0	12.0	0.788617	0.005104	0.844156	0.010146	0.788545	0.929018	0.267008
1	2	64.0	49677.0	10246.0	13.0	0.829017	0.006208	0.831169	0.012323	0.829014	0.924480	0.269911
2	3	72.0	49500.0	10423.0	5.0	0.826200	0.006860	0.935065	0.013621	0.826060	0.946679	0.146193
3	4	74.0	48551.0	11371.0	4.0	0.810417	0.006466	0.948718	0.012844	0.810237	0.956182	0.102656
4	5	67.0	46958.0	12964.0	11.0	0.783750	0.005142	0.858974	0.010222	0.783652	0.935413	0.321434
5	Mean	68.4	48387.6	11535.0	9.0	0.807600	0.005956	0.883616	0.011831	0.807502	0.938354	0.221441

Фиг. 47 Резултати от Логистична регресия



Фиг. 49 Тегло на характеристиките за Логистична регресия

Обобщение на производителността

Undersampling при Logistic Regression дава най-лошите резултати от всички LR конфигурации. Recall е ~88% — от ~77 измами на fold улавя ~68 и пропуска ~9.

По-ниско от SMOTE/class_weight (~93%), но все още високо.

Precision е ~0,6% — от ~11 603 аларми на fold (~68 TP + ~11 535 FP) само ~68 са реални измами. 99,4% от алармите са фалшиви — по-лошо от SMOTE (~2%) и class_weight (~2%). False positive достигат ~11 535 на fold — ~19% от legit транзакциите се маркират погрешно.

F1 е 0,012 — practically нулев. PR AUC 0,22 е най-ниската за LR. Accuracy пада на ~80,8% — един от малкото случаи, в които accuracy ясно показва проблема.

LR + undersample е най-лошият вариант в случая — хиляди повече false positive от SMOTE/class_weight, при по-нисък recall.

Feature importances — какво „вижда“ моделът

Доминират type_CASH_IN (~2,7), errorBalanceDest (~2,3), oldbalanceOrg (~1,8) и type_TRANSFER (~1,5). За разлика от SMOTE/class_weight, коефициентите са по-умерени (2–3 вместо 50–84), но моделът все още маркира масово транзакции — train set от ~620 записа не е достатъчен за разграничаване на legit patterns на ~60 000 test транзакции.

За Logistic Regression undersample е по-лош от SMOTE и class_weight — най-много false positive при по-нисък recall. Стратегията не е подходяща за този модел и dataset.

Резултати от Support Vector Machine (SVM) със стратегия undersample

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	65.0	51032.0	8891.0	12.0	0.851617	0.007258	0.844156	0.014392	0.851626	0.943255	0.207147
1	2	66.0	52773.0	7150.0	11.0	0.880650	0.009146	0.857143	0.018100	0.880680	0.947539	0.281007
2	3	72.0	53247.0	6676.0	5.0	0.888650	0.010670	0.935065	0.021099	0.888590	0.965328	0.390710
3	4	75.0	53267.0	6655.0	3.0	0.889033	0.011144	0.961538	0.022033	0.888939	0.973403	0.052261
4	5	68.0	51512.0	8410.0	10.0	0.859667	0.008021	0.871795	0.015895	0.859651	0.951534	0.306621
5	Mean	69.2	52366.2	7556.4	8.2	0.873923	0.009248	0.893939	0.018304	0.873897	0.956212	0.247549

Фиг. 50 Резултати от SVM

Обобщение на производителността

Undersampling при SVM дава лош trade-off, подобен на Logistic Regression. Recall е ~89% — от ~77 измами на fold улавя ~69 и пропуска ~8. По-ниско от SMOTE/class_weight (~91–93%), но все още високо.

Precision е ~0,9% — от ~7 565 аларми на fold (~69 TP + ~7 556 FP) само ~69 са реални измами. 99,1% от алармите са фалшиви. False positive достигат ~7 556 на fold — ~12,6% от legit транзакциите — по-малко от LR + undersample (~11 535), но повече от SMOTE/class_weight (~3 700–4 000).

F1 е 0,018, PR AUC 0,25 — по-лоши от none (F1 0,32, PR AUC 0,44). Accuracy пада на ~87,4%.

Какво „вижда“ моделът

В notebook-а няма графика за feature importances за SVM. LinearSVC се обучава на ~620 балансиран запис — моделът „вижда“ твърде малко legit patterns и на test set от ~60 000 транзакции маркира масово като изми, както при LR + undersample.

За SVM undersample е по-лош от SMOTE и class_weight — повече false positive при по-нисък recall. none остава единственият разумен вариант сред линейните модели без threshold tuning.

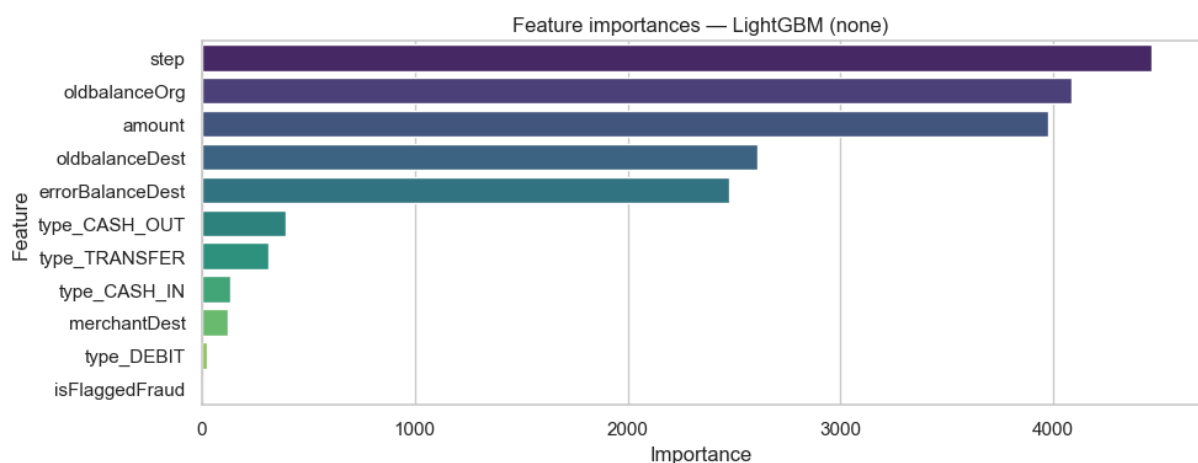
Gradient Boosting (LightGBM, XGBoost)

LightGBM и XGBoost са tree-based boosting модели — последователно надграждат ensemble от decision trees, като всяко ново дърво коригира грешките на предишните. За разлика от Random Forest (parallel bagging), boosting често постига по-добро ранжиране и по-фини decision boundaries. И двата модела се оценяват със същите четири стратегии за дисбаланс; при class_weight дисбалансът се коригира чрез scale_pos_weight ≈ 774 (ratio legit/fraud), а не чрез class_weight="balanced". LightGBM и XGBoost са конфигурирани с learning rate 0.05 и 300–400 estimators, без scaling на features. В резултатите XGBoost е сред най-добрите модели — особено с none (recall $\sim 73\%$, precision $\sim 97\%$, F1 $\sim 0,83$, PR AUC $\sim 0,86$) и class_weight (recall $\sim 85\%$, precision $\sim 55\%$, F1 $\sim 0,67$). LightGBM е по-нестабилен: силен при SMOTE/undersample по recall, но слаб при none и class_weight.

Резултати от LightGBM със стратегия none

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
	0	1	32.0	59811.0	112.0	45.0	0.997383	0.222222	0.415584	0.289593	0.998131	0.194956
	1	2	56.0	59908.0	15.0	21.0	0.999400	0.788732	0.727273	0.756757	0.999750	0.740398
	2	3	48.0	59916.0	7.0	29.0	0.999400	0.872727	0.623377	0.727273	0.999883	0.764583
	3	4	46.0	59848.0	74.0	32.0	0.998233	0.383333	0.589744	0.464646	0.998765	0.362050
	4	5	41.0	59837.0	85.0	37.0	0.997967	0.325397	0.525641	0.401961	0.998581	0.330390
	5	Mean	44.6	59864.0	58.6	32.8	0.998477	0.518482	0.576324	0.528046	0.999022	0.478476

Фиг. 51 Резултати от LightGBM



Фиг. 52 Тегло на характеристиките за LightGBM

Обобщение на производителността

LightGBM без корекция за дисбаланс показва по-слаби и по-небалансиран резултати от Random Forest и XGBoost при същата стратегия. Recall е ~58% — от ~77 измами на fold улавя ~45 и пропуска ~33. Precision е ~52% — когато моделът каже „измама“, е прав в половината от случаите. F1 0,53 и PR AUC 0,48 са значително по-ниски от RF (~0,81 / 0,81) и XGBoost (~0,83 / 0,86).

False positive са ~59 на fold — много повече от RF (~1–2) и XGBoost, но далеч от thousands при SMOTE/undersampling при линейни модели. Accuracy остава ~99,85% и отново е подвеждаща. ROC AUC 0,88 е най-нисък сред tree-based моделите с none.

Моделът е умерено агресивен — не е толкова консервативен като RF, но и не улавя достатъчно измами. Trade-off-ът между precision и recall е почти 50/50, което при fraud detection рядко е достатъчно добро без допълнителен tuning.

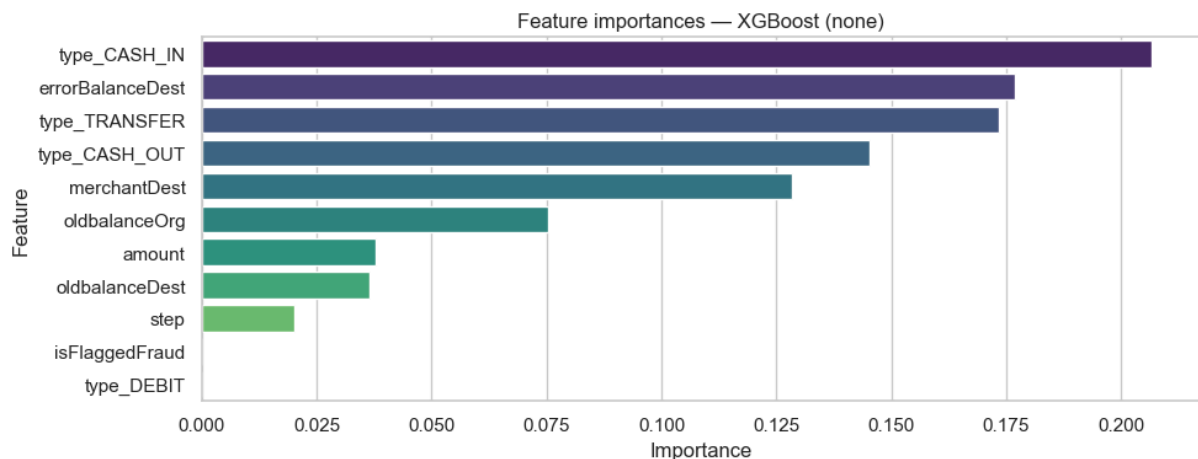
Feature importances — какво „вижда“ моделът

LightGBM разчита на step (~4500), oldbalanceOrg (~4100) и amount (~4000) — времеви индекс и балансови features, не errorBalanceDest като при RF. Следват oldbalanceDest и errorBalanceDest (~2500). Типовете транзакции и isFlaggedFraud имат по-малко влияние. Профилът е различен от Random Forest — LightGBM „вижда“ измама през време и сума, а не primarily чрез балансови несъответствия.

Резултати от XGBoost със стратегия none

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	54.0	59922.0	1.0	23.0	0.999600	0.981818	0.701299	0.818182	0.999983	0.998937	0.840567
1	2	55.0	59923.0	0.0	22.0	0.999633	1.000000	0.714286	0.833333	1.000000	0.998123	0.879947
2	3	51.0	59920.0	3.0	26.0	0.999517	0.944444	0.662338	0.778626	0.999950	0.998011	0.823766
3	4	64.0	59920.0	2.0	14.0	0.999733	0.969697	0.820513	0.888889	0.999967	0.999288	0.912715
4	5	58.0	59920.0	2.0	20.0	0.999633	0.966667	0.743590	0.840580	0.999967	0.997203	0.850682
5	Mean	56.4	59921.0	1.6	21.0	0.999623	0.972525	0.728405	0.831922	0.999973	0.998312	0.861536

Фиг. 53 Резултати от XGBoost



Фиг. 54 Тегло на характеристиките за XGBoost

Обобщение на производителността

XGBoost без корекция за дисбаланс е най-добрият baseline сред всички модели в pipeline-a. Recall е ~73% — от ~77 измами на fold улавя ~56 и пропуска ~21. Precision е ~97% — когато моделът каже „измама“, почти винаги е прав. F1 0,83 и PR AUC 0,86 са най-високите при none стратегия — малко по-добри от Random Forest (F1 0,81, PR AUC 0,81).

False positive остават ~1–2 на fold, false negative ~21 — профилът е близък до RF, но с по-висок recall и по-добро ранжиране (ROC AUC 0,998). Accuracy ~99,96% отново е подвеждаща при дисбаланс.

Моделът е консервативен, но ефективен — минимум фалшиви аларми при приемливо откриване на измами. За fraud detection с приоритет precision това е най-силната конфигурация в notebook-a.

Feature importances — какво „вижда“ моделът

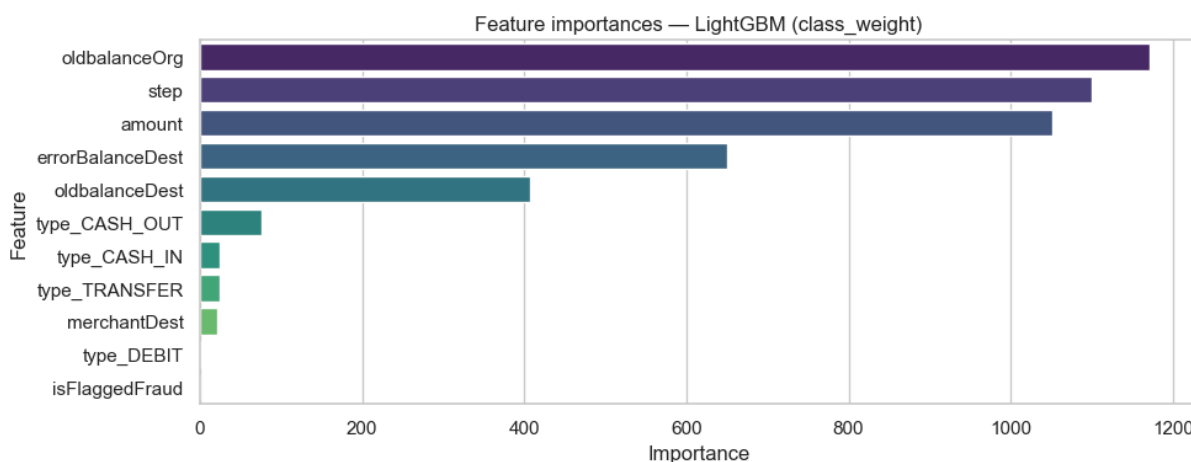
XGBoost разчита различно от RF и LightGBM. Най-важни са type_CASH_IN (~0,21), errorBalanceDest (~0,18), type_TRANSFER (~0,17) и type_CASH_OUT (~0,14), следвани от merchantDest (~0,13). oldbalanceOrg и amount имат по-малко влияние от RF; step е слаб; isFlaggedFraud и type_DEBIT са без значение.

Моделът „вижда“ измама чрез комбинация от тип транзакция, балансови несъответствия и merchant flag — по-категориален профил от RF, който domina с errorBalanceDest и баланси.

Резултати от LightGBM със стратегия class_weight

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	50.0	46154.0	13769.0	27.0	0.770067	0.003618	0.649351	0.007196	0.770222	0.709787	0.002801
1	2	48.0	55764.0	4159.0	29.0	0.930200	0.011410	0.623377	0.022409	0.930594	0.776957	0.007596
2	3	24.0	58029.0	1894.0	53.0	0.967550	0.012513	0.311688	0.024060	0.968393	0.638076	0.004783
3	4	72.0	55593.0	4329.0	6.0	0.927750	0.016360	0.923077	0.032150	0.927756	0.925411	0.015205
4	5	61.0	57058.0	2864.0	17.0	0.951983	0.020855	0.782051	0.040626	0.952205	0.867108	0.016593
5	Mean	51.0	54519.6	5403.0	26.4	0.909510	0.012951	0.657909	0.025288	0.909834	0.783468	0.009396

Фиг. 55 Резултати от LightGBM



Фиг. 56 Тегло на характеристиките за LightGBM

Обобщение на производителността

При LightGBM class_weight (чрез scale_pos_weight ≈ 774) влошава резултатите спрямо none, не ги подобрява. Recall е ~66% — от ~77 измами на fold улавя ~51 и пропуска ~26. Леко по-добър recall от none (~58%), но на unacceptable цена.

Precision е ~1,3% — от ~5 454 аларми на fold (~51 TP + ~5 403 FP) само ~51 са реални измами. 99% от алармите са фалшиви. False positive нарастват от ~59 на fold (none) до ~5 403 — ~9% от legit транзакциите.

F1 е 0,025, PR AUC 0,009 — practically нулеви, най-лошите в целия notebook. ROC AUC пада от 0,88 на 0,78. Accuracy пада на ~91%.

LightGBM + scale_pos_weight с default threshold е непрактичен — class_weight не работи за LightGBM така, както за XGBoost.

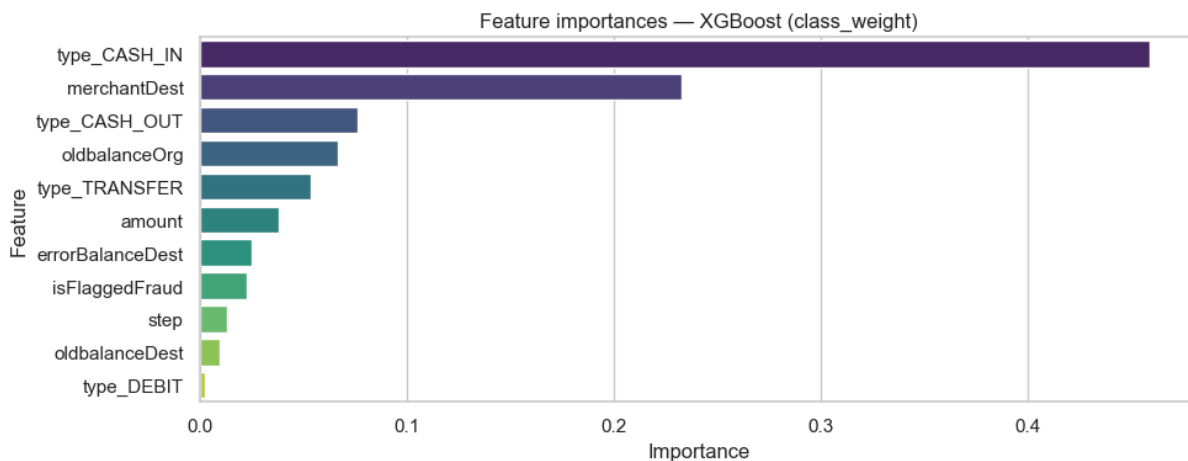
Feature importances — какво „вижда“ моделът

Профилът е близък до LightGBM none: oldbalanceOrg (~1170), step (~1100), amount (~1050), errorBalanceDest (~650). Типовете транзакции и isFlaggedFraud отново са без значение. При агресивна тежест на fraud класа моделът продължава да разчита на време и баланси, но decision boundary става твърде чувствителен.

Резултати от XGBoost със стратегия class_weight

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	64.0	59876.0	47.0	13.0	0.999000	0.576577	0.831169	0.680851	0.999216	0.998978	0.851989
1	2	68.0	59870.0	53.0	9.0	0.998967	0.561983	0.883117	0.686869	0.999116	0.998589	0.885580
2	3	62.0	59868.0	55.0	15.0	0.998833	0.529915	0.805195	0.639175	0.999082	0.997223	0.816847
3	4	70.0	59863.0	59.0	8.0	0.998883	0.542636	0.897436	0.676329	0.999015	0.999304	0.908406
4	5	66.0	59863.0	59.0	12.0	0.998817	0.528000	0.846154	0.650246	0.999015	0.997542	0.854033
5	Mean	66.0	59868.0	54.6	11.4	0.998900	0.547822	0.852614	0.666694	0.999089	0.998327	0.863371

Фиг. 57 Резултати от XGBoost



Фиг. 58 Тегло на характеристиките за XGBoost

Обобщение на производителността

При XGBoost class_weight (чрез scale_pos_weight ≈ 774) работи отлично — един от най-добрите trade-off-и в pipeline-a. Recall е ~85% — от ~77 измами на fold улавя ~66 и пропуска само ~11. Precision е ~55% — когато моделът каже „измама“, е прав в повече

от половината случаи. F1 0,67 и PR AUC 0,86 — на нивото на XGBoost none (0,83 / 0,86), но с значително по-висок recall (85% vs 73%).

False positive са ~55 на fold — повече от none (~1–2), но далеч от хилядите при LightGBM + class_weight или линейните модели. ROC AUC 0,998 остава отличен.

XGBoost + class_weight е оптимален избор, когато искаш повече открити измами без да жертваш precision напълно — контролируем trade-off, за разлика от LightGBM при същата стратегия.

Feature importances — какво „вижда“ моделът

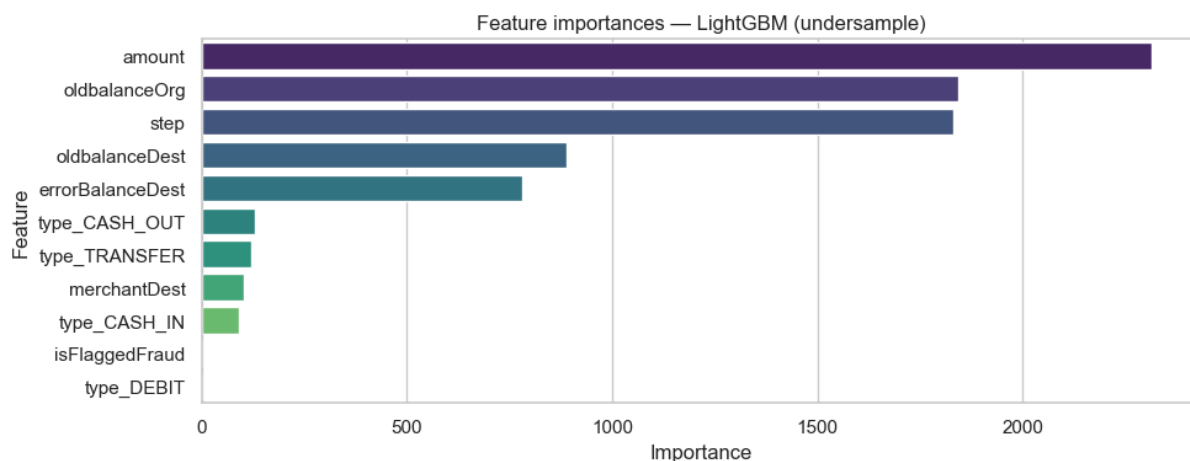
С class_weight профилът се измества към категориите характеристики. Най-важни са type_CASH_IN (~0,46) и merchantDest (~0,23), следвани от type_CASH_OUT, oldbalanceOrg и type_TRANSFER. errorBalanceDest и amount имат по-малко влияние от XGBoost none; isFlaggedFraud отново е без значение.

Моделът „вижда“ измама през тип транзакция и merchant flag, не primarily чрез балансови несъответствия — различен профил от none, където доминираха errorBalanceDest и transaction types заедно.

Резултати от LightGBM със стратегия undersample

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	76.0	57418.0	2505.0	1.0	0.958233	0.029446	0.987013	0.057186	0.958196	0.996755	0.733729
1	2	76.0	57633.0	2290.0	1.0	0.961817	0.032122	0.987013	0.062219	0.961784	0.995609	0.651907
2	3	75.0	57507.0	2416.0	2.0	0.959700	0.030108	0.974026	0.058411	0.959682	0.992209	0.684495
3	4	77.0	56724.0	3198.0	1.0	0.946683	0.023511	0.987179	0.045929	0.946631	0.996276	0.673942
4	5	76.0	57035.0	2887.0	2.0	0.951850	0.025650	0.974359	0.049984	0.951821	0.994965	0.558254
5	Mean	76.0	57263.4	2659.2	1.4	0.955657	0.028167	0.981918	0.054746	0.955623	0.995163	0.660466

Фиг. 59 Резултати от LightGBM



Фиг. 60 Тегло на характеристиките за LightGBM

Обобщение на производителността

Undersampling при LightGBM дава екстремален профил — почти максимален recall при катастрофален precision. Recall е ~98%: от ~77 измами на fold улавя ~76 и пропуска само ~1–2. Почти никоя измама не остава незабелязана.

Precision е ~2,8% — от ~2 735 аларми на fold (~76 TP + ~2 659 FP) само ~76 са реални измами. 97 от 100 аларми са фалшиви. False positive достигат ~2 659 на fold — ~4,4% от legit транзакциите.

F1 е 0,055, PR AUC 0,66 — по-лоши от LightGBM none (F1 0,53, PR AUC 0,48 за F1 no worse PR context - actually PR AUC 0.66 is better than none's 0.48 but F1 is much worse). Accuracy пада на ~95,6%. ROC AUC (0,995) е висок, но подвеждащ при такъв precision. LightGBM + undersample е непрактичен за production — същият проблем като RF + undersample: хиляди фалшиви аларми за минимален gain в recall спрямо по-умерени стратегии.

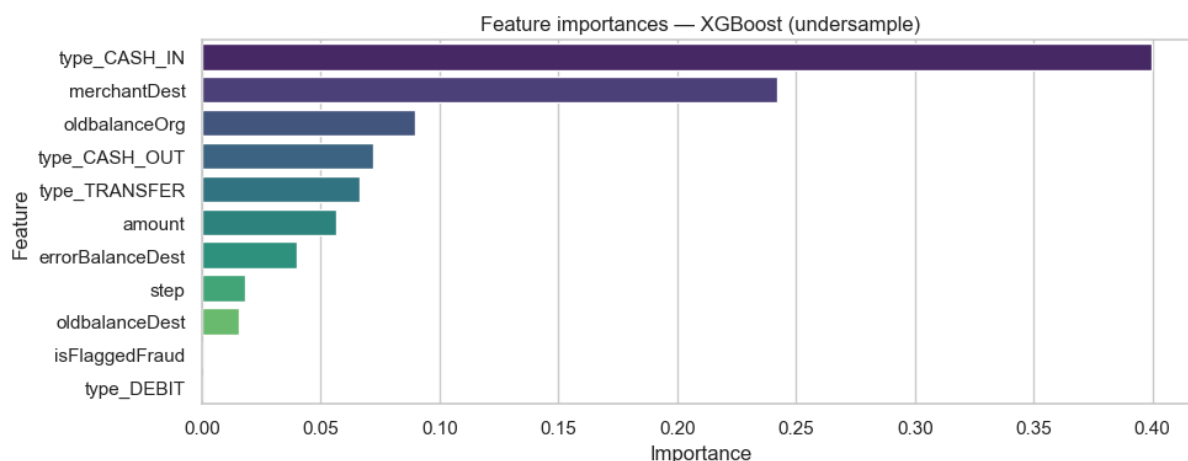
Feature importances — какво „вижда“ моделът

Доминират amount (>2000), oldbalanceOrg (~1800) и step (~1800), следвани от oldbalanceDest и errorBalanceDest. Типовете транзакции и isFlaggedFraud са без значение. Моделът се обучава на ~620 балансиран запис и „вижда“ измама през сума, време и баланси — но без достатъчно legit patterns генерира масови false positive на test.

Резултати от XGBoost със стратегия undersample

	Fold	TP	TN	FP	FN	Accuracy	Precision	Recall	F1	Specificity	ROC_AUC	PR_AUC
0	1	73.0	57274.0	2649.0	4.0	0.955783	0.026819	0.948052	0.052161	0.955793	0.995185	0.704801
1	2	76.0	57071.0	2852.0	1.0	0.952450	0.025956	0.987013	0.050582	0.952406	0.993992	0.533442
2	3	74.0	57384.0	2539.0	3.0	0.957633	0.028320	0.961039	0.055019	0.957629	0.993978	0.646980
3	4	76.0	56783.0	3139.0	2.0	0.947650	0.023639	0.974359	0.046159	0.947615	0.995901	0.712739
4	5	75.0	57154.0	2768.0	3.0	0.953817	0.026381	0.961538	0.051352	0.953807	0.994455	0.585046
5	Mean	74.8	57133.2	2789.4	2.6	0.953467	0.026223	0.966400	0.051055	0.953450	0.994702	0.636602

Фиг. 61 Резултати от XGBoost



Фиг. 62 Тегло на характеристиките за XGBoost

Обобщение на производителността

Undersampling при XGBoost дава екстреман профил — почти максимален recall при катастрофален precision, както при LightGBM и Random Forest със същата стратегия. Recall е ~97%: от ~77 измами на fold улавя ~75 и пропуска само ~2–3. Почти никоя измама не остава незабелязана.

Precision е ~2,6% — от ~2 864 аларми на fold (~75 TP + ~2 789 FP) само ~75 са реални измами. 97 от 100 аларми са фалшиви. False positive достигат ~2 789 на fold — ~4,6% от legit транзакциите.

F1 е 0,051, PR AUC 0,64 — далеч от XGBoost none (F1 0,83, PR AUC 0,86) и class_weight (F1 0,67, PR AUC 0,86). Accuracy пада на ~95,3%. ROC AUC (0,995) е висок, но подвеждащ при такъв precision.

XGBoost + undersample е непрактичен — въпреки че XGBoost е най-добрият модел в pipeline-а при други стратегии, undersampling го разрушава.

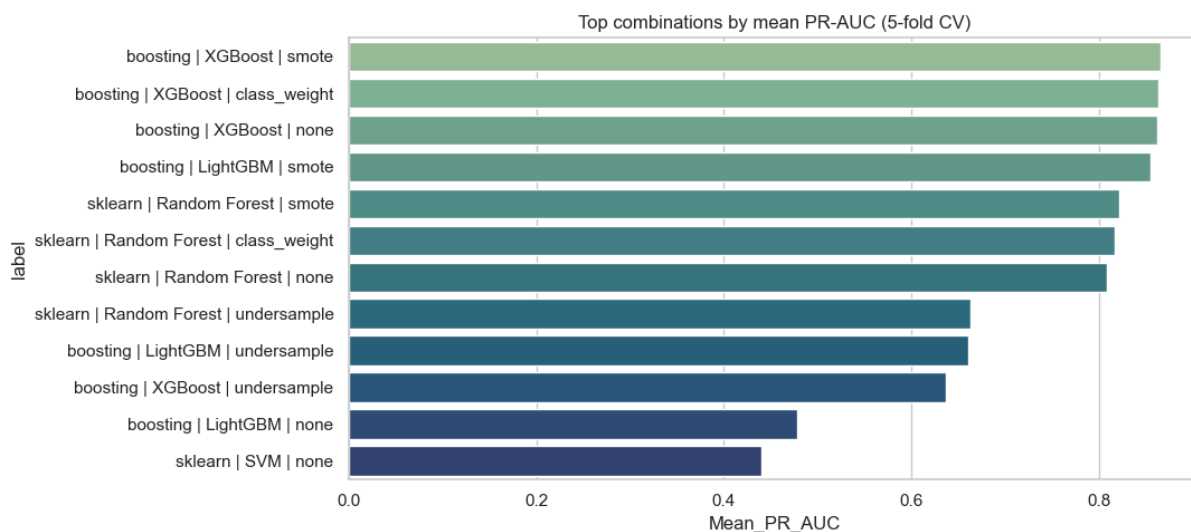
Feature importances — какво „вижда“ моделът

Доминират type_CASH_IN (~0,40) и merchantDest (~0,24), следвани от oldbalanceOrg, type_CASH_OUT и type_TRANSFER. Профилът е близък до XGBoost class_weight, не до none — моделът се „залепва“ за категорийните характеристики след обучение на малък балансиран train set. isFlaggedFraud и type_DEBIT са без значение.

Сравнителен анализ на всички модели

	Family	Model	Strategy	Mean_Recall	Mean_Precision	Mean_F1	Mean_PR_AUC	Mean_ROC_AUC
17	boosting	XGBoost	smote	0.938029	0.195953	0.323872	0.865041	0.997770
15	boosting	XGBoost	class_weight	0.852614	0.547822	0.666694	0.863371	0.998327
13	boosting	XGBoost	none	0.728405	0.972525	0.831922	0.861536	0.998312
16	boosting	LightGBM	smote	0.883650	0.370890	0.522031	0.854545	0.996324
7	sklearn	Random Forest	smote	0.834366	0.481912	0.610499	0.820911	0.993319
4	sklearn	Random Forest	class_weight	0.692208	0.974788	0.808261	0.816184	0.965124
1	sklearn	Random Forest	none	0.699933	0.972013	0.812215	0.808043	0.959335
10	sklearn	Random Forest	undersample	0.968931	0.026719	0.052002	0.662403	0.994898
18	boosting	LightGBM	undersample	0.981918	0.028167	0.054746	0.660466	0.995163
19	boosting	XGBoost	undersample	0.966400	0.026223	0.051055	0.636602	0.994702
12	boosting	LightGBM	none	0.576324	0.518482	0.528046	0.478476	0.876981
2	sklearn	SVM	none	0.193839	0.906774	0.317243	0.439607	0.961077
0	sklearn	Logistic Regression	none	0.248019	0.899712	0.385973	0.396191	0.939778
8	sklearn	SVM	smote	0.914685	0.018579	0.036418	0.285457	0.978026
6	sklearn	Logistic Regression	smote	0.932801	0.019318	0.037852	0.253286	0.980434
5	sklearn	SVM	class_weight	0.925008	0.017682	0.034700	0.248720	0.979052
11	sklearn	SVM	undersample	0.893939	0.009248	0.018304	0.247549	0.956212
3	sklearn	Logistic Regression	class_weight	0.930203	0.017339	0.034042	0.227268	0.980136
9	sklearn	Logistic Regression	undersample	0.883616	0.005956	0.011831	0.221441	0.938354
14	boosting	LightGBM	class_weight	0.657909	0.012951	0.025288	0.009396	0.783468

Фиг. 63 Сравнителен анализ на моделите



Фиг. 64 Сравнение на най-добрите комбинации алгоритм × стратегия за небалансирани класове по метрика Mean PR-AUC

Сравнението на 20 комбинации модел × стратегия (сортирани по Mean_PR_AUC и Mean_Recall) показва ясна йерархия. Tree-based моделите (XGBoost, Random Forest, LightGBM) значително надминават линейните (Logistic Regression, SVM) — последните при none имат recall 19–25% и PR AUC под 0,44, докато най-добрите tree конфигурации достигат PR AUC 0,81–0,87. XGBoost доминира топ 3 по PR AUC: smote (0,865, recall 94%), class_weight (0,863, recall 85%, precision 55%) и none (0,862, recall 73%, precision 97%, F1 0,832 — най-висок в целия експеримент). Random Forest е стабилен при none и

class_weight (F1 ~0,81, precision ~97%), но class_weight не добавя стойност спрямо baseline; SMOTE повишава recall до 83% при precision 48%. Стратегиите SMOTE, class_weight и undersample при линейните модели дават recall 88–93%, но precision 0,6–2% и F1 под 0,04 — непрактични при default threshold. Undersampling е най-лошият trade-off: recall 97–98%, но precision 2–3% и F1 ~0,05, дори при XGBoost. LightGBM е нестабилен — силен при SMOTE/undersample по recall, но слаб при none (PR AUC 0,48) и катастрофален при class_weight (PR AUC 0,009). ROC AUC е висок почти навсякъде (0,94–0,99), но при дисбаланс PR AUC и F1 разкриват реалната картина по-надеждно от accuracy.

ЗАКЛЮЧЕНИЕ

Целта на изследването беше да се оцени потенциалът на алгоритми за машинно обучение за детекция на финансови измами върху PaySim набор от данни. При такъв тип задачи ранното откриване на подозрителни транзакции е от критична важност — пропуснатата измама носи директни финансови загуби, докато фалшивите аларми водят до прекъснати легитимни операции и загуба на доверие. Поради това адаптирането и интегрирането на ML в транзакционни системи би повишило ефикасността на fraud detection, намалявайки загубите и подобрявайки защитата на клиентите.

В хода на изследването се отличиха няколко основни момента. Разглеждането на структурата на данните помогна за разбиране на силния класов дисбаланс (~0,13% измами). Бяха създадени инженерни характеристики (errorBalanceDest, merchantDest), а корелационният анализ и PCA допринесоха за по-добро разбиране на набора от характеристики и премахване на излишни променливи. Предварителната обработка подготви данните за моделите — one-hot encoding на типа транзакция и stratified 5-fold cross-validation с коректно прилагане на resampling само върху train fold, за да се избегне загуба на данни. За експериментите бяха използвани пет модела (Logistic Regression, Random Forest, SVM, LightGBM, XGBoost) и четири стратегии за небалансирани класове (none, class_weight, SMOTE, undersampling) — общо 20 конфигурации.

При анализа на резултатите ясно се наблюдава силно разделяне между базираните на дървета и линейни модели. XGBoost и Random Forest постигнаха най-добри резултати по PR AUC (0,81–0,87) и F1 (до 0,83), докато Logistic Regression и SVM при none имаха recall 19–25% и пропускаха над 75% от измамите. Accuracy (~99,9%) беше подвеждаща за всички конфигурации — следствие от дисбаланса. PR AUC и F1 се оказаха по-информативни метрики. Стратегиите SMOTE, class_weight и undersampling при линейните модели повишиха recall до 88–93%, но precision паднаше до 0,6–2%, което го правеше практически неприложимо. Undersampling даде recall 97–98%, но precision 2–3% и F1 ~0,05. При Random Forest class_weight не добави стойност спрямо none, докато SMOTE повиши recall до 83% при precision 48%. XGBoost се отличи като най-стабилен модел: none (recall 73%, precision 97%, F1 0,83), class_weight (recall 85%, precision 55%) и SMOTE (recall 94%, PR AUC 0,865).

Наличието на толкова добри резултати при базираните на дървета модели, особено по характеристиките свързани с баланс (errorBalanceDest, oldbalanceOrg), най-вероятно се

обяснява със синтетичния характер на PaySim — измамите в симулацията следват предсказуеми patterns, които real-world fraud може да не повторя. isFlaggedFraud имаше нулево влияние, което показва ограниченията на набора от данни. В реална банкова среда липсват поведенчески характеристики (velocity, device fingerprint, network analysis), а моделите на измама се променят непрекъснато. Високият ROC AUC (0,94–0,99) при някои слаби конфигурации подсилва нуждата от PR AUC и confusion matrix, не само от ROC.

Следващи стъпки биха били threshold tuning и cost-sensitive evaluation според бизнес разходите FN vs FP; тестване върху реални транзакционни данни; добавяне на поведенчески и мрежови характеристики; и deployment на XGBoost + none или class_weight според приоритета на организацията — минимум фалшиво позитивни или максимум открити измами.

Работата демонстрира потенциала на машинното обучение за откриване на измами при небалансирани данни, но показва и колко критични са изборът на модел, стратегията за дисбаланс и качеството на данните за надеждна интеграция в реалния свят.



**ТЕХНИЧЕСКИ УНИВЕРСИТЕТ –
СОФИЯ** Факултет Приложна математика и
информатика

**ДЕКЛАРАЦИЯ
ЗА АВТОРСТВО НА ДИПЛОМНА РАБОТА**

Долуподписаният/ата Тодор Георгиев Стаматов
(име, презиме, фамилия)

специалност: Анализ на големи масиви и потоци данни фак. № 961324007

ДЕКЛАРИРАМ:

Представената от мен дипломна работа на тема:
Методи за детекция на финансови измами чрез машинно обучение
при небалансирани данни

е лична моя авторска разработка, резултат от собствени изследвания.
Потвърждавам, че тя в нейната цялост и отделни части не е била използвана
за придобиване на образователна и/или научна степен в ТУ - София или в
други университети.

Формулировки, идеи и текстове, взети от други източници, са цитирани
точно и с коректно посочване на техните автори. Дипломната работа не е
публикувана на друго място.

Декларирам, че предоставям правото на Факултет Приложна
математика и информатика при ТУ - София, съгласно процедурите и
правилниците на университета, да архивира и съхранява тази дипломна
работа с цел доказване във времето на моето авторство.

Дата: 2026 г.

Дипломант:

София

ИЗПОЛЗВАНИ ЛИТЕРАТУРНИ ИЗТОЧНИЦИ

1. PaySim — основна научна публикация Lopez-Rojas, E. A., Elmir, A., & Axelsson, S. (2016). PaySim: A financial mobile money simulator for fraud detection. EMSS 2016, pp. 249–255. Semantic Scholar:
<https://www.semanticscholar.org/paper/Paysim:-a-financial-mobile-money-simulator-for-Lopez-Rojas-Elmir/3a1d2057543cdf2a771f7421c8deab280a04779d>
2. Официално PaySim GitHub репозитори
<https://github.com/EdgarLopezPhD/PaySim>
3. Kaggle dataset <https://www.kaggle.com/datasets/ealaxi/paysim1>
4. Hugging Face dataset <https://huggingface.co/datasets/theman10/paysim>
5. Chawla, N. V., et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR, 16, 321–357.
<https://arxiv.org/abs/1106.1813>
<https://jair.org/index.php/jair/article/view/10302>
6. Lemaître, G., et al. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. JMLR, 18(17), 1–5.
 - a. <https://jmlr.org/papers/v18/16-003.html>
 - b. Документация: <https://imbalanced-learn.org/>
7. Random Forest: Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32. <https://link.springer.com/article/10.1023/A:1010933404324>
8. XGBoost: Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.
 - a. <https://github.com/dmlc/xgboost>
 - b. <https://dl.acm.org/doi/10.1145/2939672.2939785>
9. LightGBM: Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.
 - a. <https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree>
 - b. <https://github.com/microsoft/LightGBM>
10. Logistic Regression (sklearn реализация)
 - a. scikit-learn документация:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
11. SVM / LinearSVC: Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. Machine Learning, 20(3), 273–297.:
<https://link.springer.com/article/10.1007/BF00994018>
12. PR AUC при небалансирани данни: Davis, J., & Goadrich, M. (2006). The Relationship Between Precision-Recall and ROC Curves. ICML 2006.
 - a. <https://dl.acm.org/doi/10.5555/1143844.1143874>
13. scikit-learn — метрики и StratifiedKFold

- a. https://scikit-learn.org/stable/modules/model_evaluation.html
- b. https://scikit-learn.org/stable/modules/cross_validation.html#stratified-k-fold

14.

ИЗХОДЕН КОД НА ИЗСЛЕДВАНЕТО

GitHub - TBD